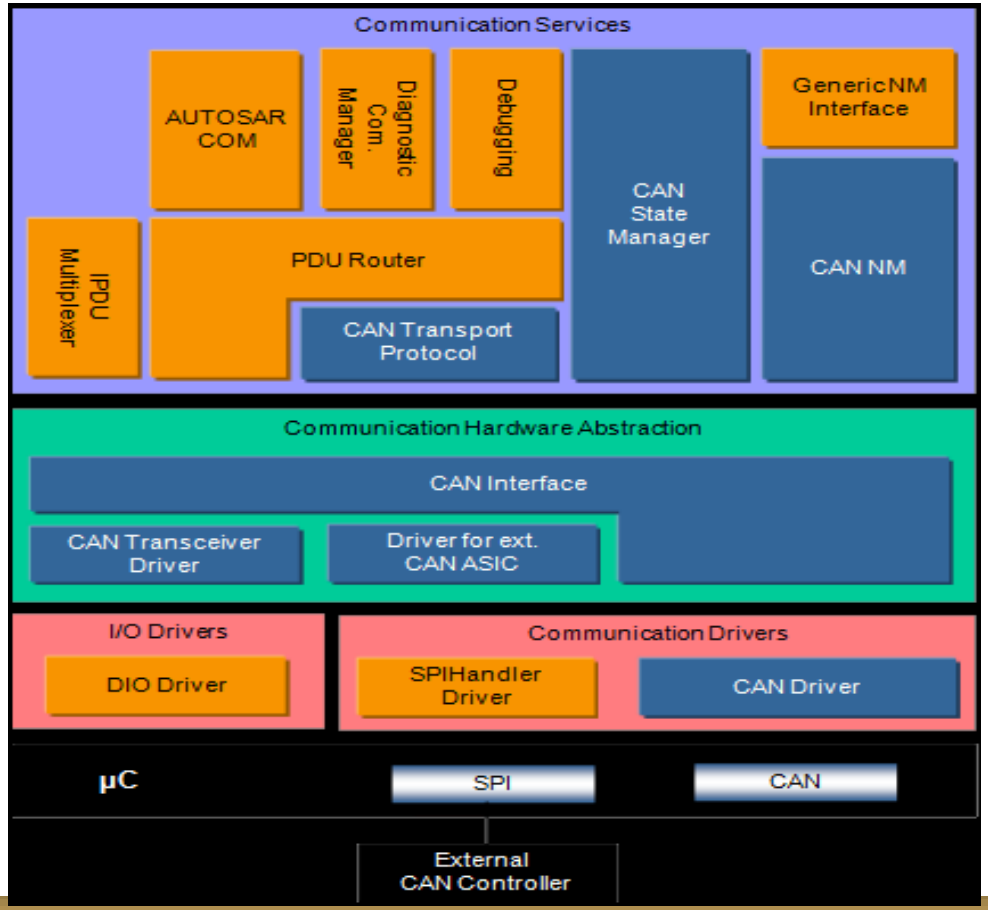
AUTOSAR COM Stack

Agenda

- Introduction
- CAN
- CAN Driver Module
- CAN IF Module
- CanTP Module
- PduR Module
- COM Module

Can Interface



- The CAN Interface module is located between the low level CAN device drivers (CAN Driver and Transceiver Driver) and the upper communication service layers (i.e. CAN State Manager , CAN Network Management , CAN Transport Protocol , PDU Router).

- The CAN Interface module provides a unique interface to manage different CAN hardware device types like CAN Controllers and CAN Transceivers used by the defined ECU hardware layout.

- The CAN Interface module consists of all CAN hardware independent tasks.

- CanIf fulfils main control flow and data flow requirements of the PDU Router and upper layer communication modules of the AUTOSAR COM stack*:*

➢ *transmit request processing*
➢ *transmit confirmation*
➢ *receive indication*
➢ *error notification*
➢ *start / stop of a CAN Controller and thus waking up / participating on a network.*

- Its data processing and notification API is based on CAN L-SDUs, whereas APIs for control and mode handling provides a CAN Controller related view.

- In case of Transmit Requests CanIf completes the L-PDU transmission with corresponding parameters and relays the CAN L-PDU via the appropriate CanDrv to the CAN Controller.

- At reception CanIf distributes the Received L-PDUs as L-SDUs to the upper layer. The assignment between Receive L-SDU and upper layer is statically configured. At transmit confirmation CanIf is responsible for the notification of upper layers about successful transmission.

- The CAN Interface forwards downwards the status change requests from the CAN State Manager to the lower layer CAN device drivers, and upwards the CAN Driver / CAN Transceiver Driver events are forwarded by the CAN Interface module to e.g. the corresponding NM module.

Important Definitions

- ***CAN L-PDU*** *:* CAN Protocol Data Unit. Consists of an identifier, Data Length and data (SDU) Visible to the CAN driver.

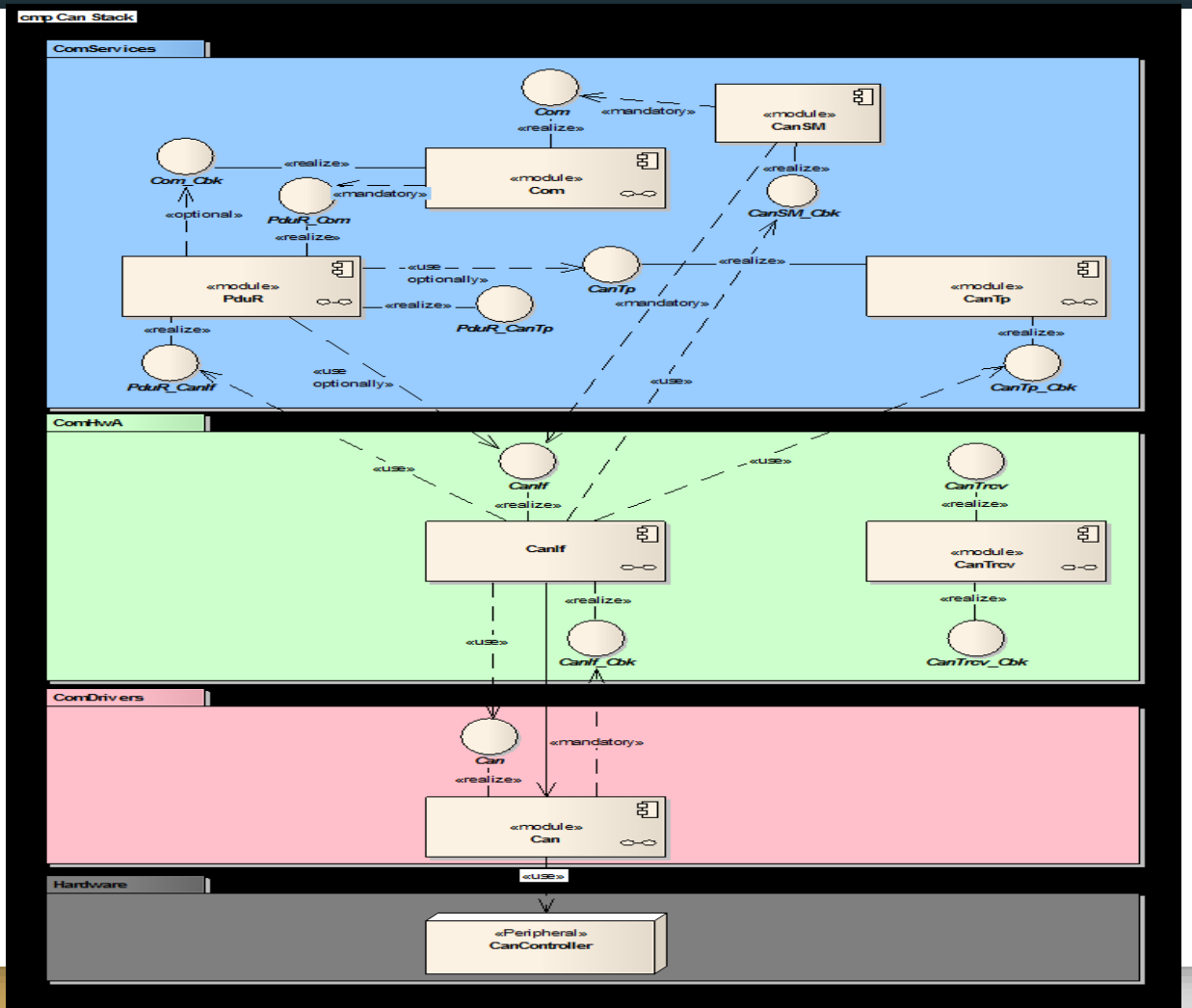
- ***CAN L-SDU*** **:** CAN Service Data Unit. Data that are transported inside the CAN L-PDU. Visible to the upper layers of the CAN interface (e.g. PDU Router).

- ***Hardware Receive Handle (HRH) :*** The Hardware Receive Handle (HRH) is defined and provided by the CAN Driver. Each HRH typically represents just one hardware object. The HRH is used as a parameter by the CAN Interface Layer for i.e. software filtering.

- ***Hardware Transmit Handle (HTH) :*** The Hardware Transmit Handle (HTH) is defined and provided by the CAN Driver. Each HTH typically represents just one or multiple CAN hardware objects that are configured as CAN hardware transmit buffer pool.

- ***Physical channel:*** A physical channel represents an interface from a CAN Controller to the CAN Network. Different physical channels of the CAN Hardware Unit may access different networks.

Can If Interaction with other modules



Initialization: Ecu State Manager

Mode Control: CAN State Manager

Lower layers: CAN Driver

- CanDrv provides a normalized L-PDU to ensure hardware independence of CanIf. The pointer to this normalized L-PDU points either to a temporary buffer (for e.g. data normalizing) or to the CAN hardware dependent CanDrv. For CanIf the kind of L-PDU buffer is invisible.

- The buffer location is passed by the CanIf from or to the CAN Driver module(CanDrv) during transmission and reception. During execution of these transmission/ reception indication services buffer location is passed. Data integrity is guaranteed by use of lock mechanisms each time the buffer has been accessed.

- The CanIf provides notification services used by the CanDrv in all notifications scenarios:
  - transmit confirmation (CanIf_TxConfirmation)
  - receive indication (CanIf_RxIndication)
  - notification of a controller mode change

Lower layers: CAN Transceiver Driver

The CanIf maps the following services of all underlying CanTrcvs to one unique interface.

- Unique CanTrcv mode request and read services to manage the operation modes of each underlying CAN Transceiver device.
- Read service for CAN Transceiver wake up reason support.
- Mode request service to enable/disable/clear wake up event state of each used CAN transceiver (CanIf_SetTrcvMode())

General Functionality of Can If

- Initialization
- Transmit request services
- Transmit confirmation services
- Reception indication services
- Controller mode control services
- PDU mode control services

Possible applications of Can If:

- ***Interrupt Mode*** :
  CanDrv processes interrupts triggered by the CAN Controller. CanIf, which is event based, is notified when an event occurs. In this case the relevant CanIf services are called within the corresponding ISRs in CanDrv.

- ***Polling Mode****:*
  CanDrv is triggered by the SchM and performs subsequent processes (Polling Mode). In this case Can_MainFunction_<Write/Read/BusOff/Wakeup/ Transceiver>() must be called periodically within a defined time interval. CanIf is notified by CanDrv about events (Reception, Transmission, BusOff, Timeout), that occurred in one of the CAN Controllers, equally to the interrupt driven operation. CanDrv is responsible for the update of the corresponding information which belongs to the occurred event in the CAN Controller, for example reception of a L-PDU.

- ***Mixed Mode***:
  Interrupt and polling driven CanDrv
  The functionality can be divided between interrupt driven and polling driven operation mode depending on the used CAN Controllers. Examples: Polling driven FullCAN reception and interrupt driven BasicCAN reception, polling driven transmit and interrupt driven reception, etc.

Hardware object handles

- Hardware Object Handles (HOH) for transmission (HTH) as well as for reception (HRH) represent an abstract reference to a CAN mailbox structure, that contains CAN related parameters such as CanId, DLC and data.
- Hardware Objects are referenced in CanIf independent of the CAN hardware buffer layout.
- The HOH is used as a parameter in the calls of CanDrv's interface services and is provided by CanDrv's configuration and used by CanDrv as identifier for communication buffers of the CAN mailbox.
- CanIf acts only as user of the Hardware Object Handle, but does not interpret it on the basis of hardware specific information.
- CanIf shall avoid direct access to hardware specific communication buffers and shall access it exclusively via CanDrv interface services.
- CanIf remains independent of hardware, because CanDrv interfaces are called with HOH parameters, which abstract from the concrete CAN hardware buffer properties.

- Each CAN Controller can provide multiple CAN Transmit Hardware Objects in the CAN mailbox. These can be logically linked to one entire pool of Hardware Objects (multiplexed Hardware Objects) and thus addressed by one HTH.

CanIf shall use two types of HOHs to enable access to Can-Drv:

- ❑ Hardware Transmit Handle (HTH)
- ❑ Hardware Receive Handle (HRH).

HRH

- The HRH shall enable CanIf to use BasicCAN or a FullCAN reception method of the referenced reception unit and to indicate a Received L-SDU to a target upper layer module.

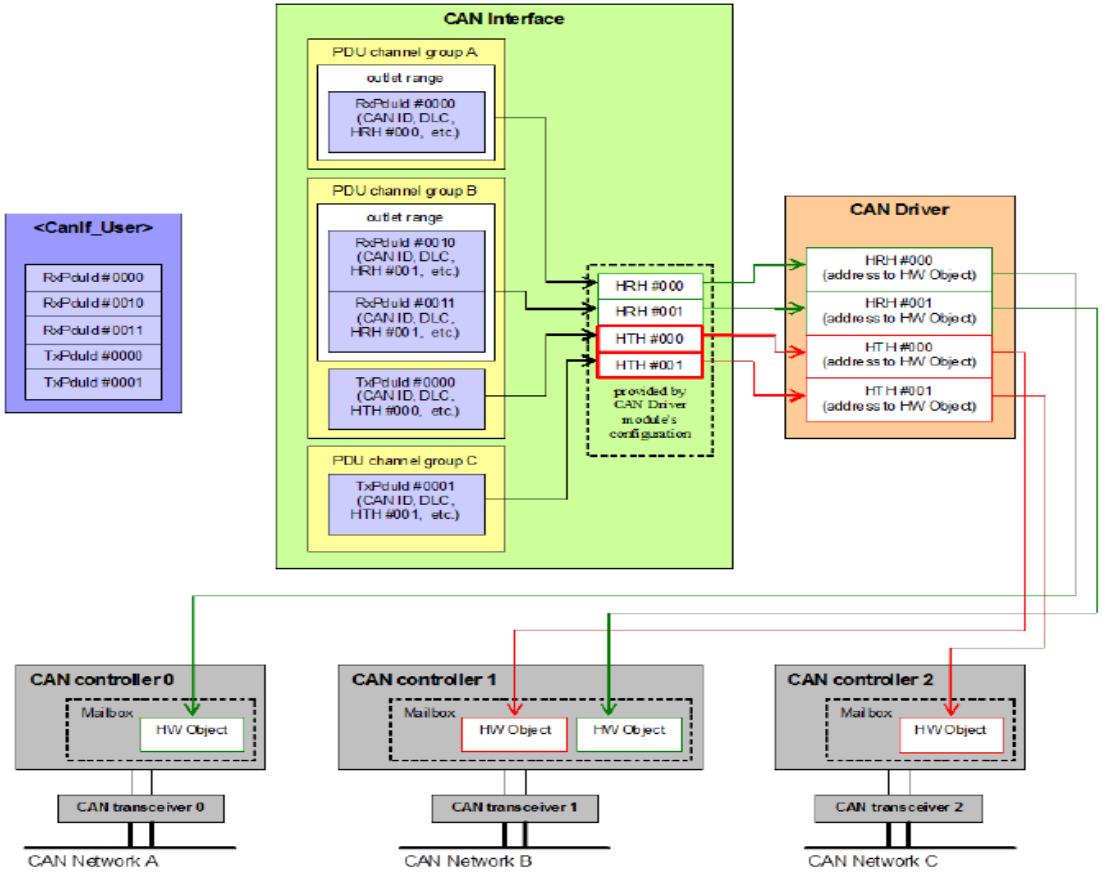
- If multiple HRHs are used, each HRH shall belong at least to a single or fixed group of Rx L-SDU (CanRxPduIds).

The HRH can be configured to receive

- one single CanId (FullCAN)
- a group of single CanIds (BasicCAN)
- a range/area of CanIds (BasicCAN) or
- all CanIds.

HTH

- The HTH shall be a handle referencing a logical Hardware Transmit Object of the CAN Controller mailbox.
- The HTH shall enable CanIf to use BasicCAN or FullCAN transmission method of the referenced transmission unit and to confirm a transmitted L-SDU to a target upper layer module.
- Each CanIf Tx L-PDU shall statically be assigned to one CanIfBufferCfg configuration container at configuration time
- If multiple HTHs are used, each HTH shall belong to a single or fixed group of Tx L-PDU (CanTxPduIds).
- CanIf shall be able to use all HRHs and HTHs of one CanDrv as common, single numbering area starting with zero.
- The dedicated HRHs and HTHs are derived from the configuration set of CanDrv. The definition of HTH/HRH inside the numbering area and Hardware Objects is up to CanDrv.



All arrows within this picture are references and no communication directions of sth. else.

Descriptions:
Outlet range= Range of Rx L-PDUs which will be passed
Mailbox = CAN RAM structure
HWObject = CAN RAM structure that contains (CanId, DLC, data)
HRH = abstract reference to the CAN RAM structure
Transmit path is coloured red
Receive path is coloured green.

Figure 7.1: Mapping between PDU Ids and HW object handles

Static L-PDUs

CanIf offers general access to the CAN L-SDU related data for upper layers.

CAN Interface specific attributes	CAN Protocol Control Information (PCI)
Method of SW filtering CanIfPrivateSoftwareFilterType	CAN Identifier (CanId) CanIfTxPduCanId, range of CanIds per *PDU* (see CanIfRxPduCanIdRange), CanIfRxPduCanId, CanIfRxPduCanIdMask
Direction of L-PDU (Tx, Rx) CanIfTxPduId, CanIfRxPduId)	Type of CAN Identifier (*StandardCAN*, *ExtendedCAN*) referenced from CanDrv via CanIfHthIdSymRef, CanIfHrhIdSymRef
HTH/HRH of the CAN Controller	Data Length and Data Length Code (*DLC*) CanIfRxPduDataLength
Target ID for the corresponding upper layer CanIfTxPduUserTxConfirmationUL, CanIfRxPduUserRxIndicationUL	Reference to the PDU data (see [1, Specification of CAN Driver])
Type of Transmit L-PDU (STATIC, DYNAMIC) CanIfTxPduType	
Type of Tx/Rx L-PDU (*FullCAN*, *BasicCAN*) CanIfHthIdSymRef, CanIfHrhIdSymRef	

Usage of L-PDUs

- For transmission request and transmission/reception polling API the upper layer module uses the L-SDU ID (CanTxPduId/CanRxPduId) defined by CanIf as parameter.

- For all callback APIs, which are invoked by CanIf at upper layer modules, CanIf passes the target PduId defined by each upper layer module as parameter.

The principle is that the caller must use the defined target L-PDU/L-SDU Id of the callee.

Dynamic L-PDUs

Dynamic Transmit L-PDUs

- L-PDUs which allow reconfiguration of the CanId during runtime (CanIfTxPduType) or where the ID or parts thereof are provided as MetaData of the L-SDU.

The usage of all other L-PDU elements are equal to normal static Transmit L-PDUs:

- The transmit confirmation notification CanIfTxPduUserTxConfirmationUL cannot be reconfigured as it belongs to the LPDU.
- The Data Length and the pointer to the data buffer are both determined by the upper layer module at call of CanIf_Transmit().

- The function CanIf_SetDynamicTxId() reconfigures the CanId of a dynamic L-PDU with CanIfTxPduType.

- CanIf shall **process the two most significant bits of the CanId** to determine which type of CanId is used and thus how the dynamic Transmit L-PDU shall be transmitted.

Dynamic receive L-PDUs

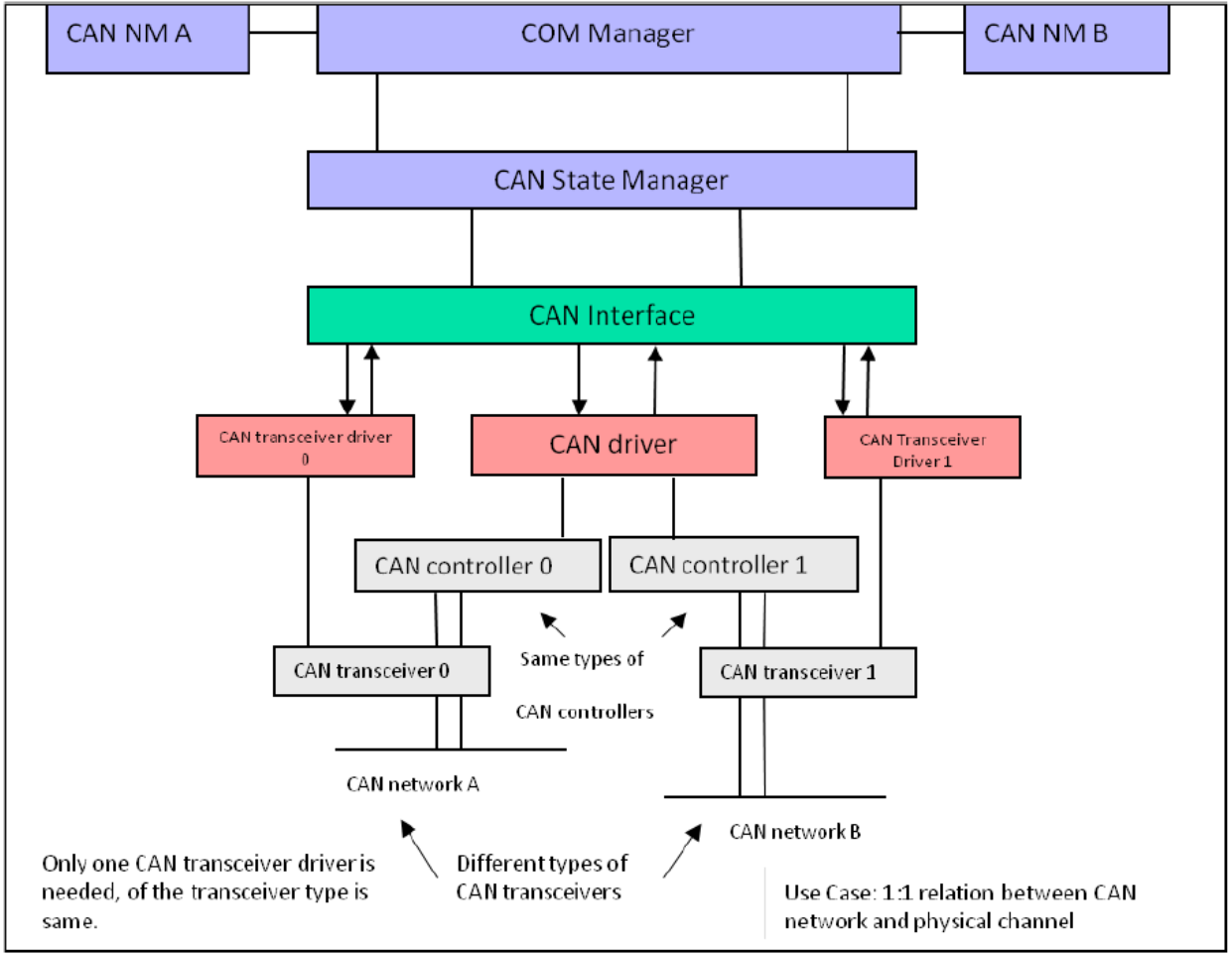
- L-PDUs that correspond to a set of CanIds, where the actually received CanId is provided to upper layers as part of the PDU data.
- Configuration shall ensure that dynamic Receive L-PDUs use an ID range or a mask and that the MetaDataItem CAN_ID_32 is configured for the L-SDU. Besides, the software filtering must be enabled for these L-SDUs.
- Upon reception of a dynamic L-SDU, CanIf shall place the CanId in the MetaDataItem of type CAN_ID_32.

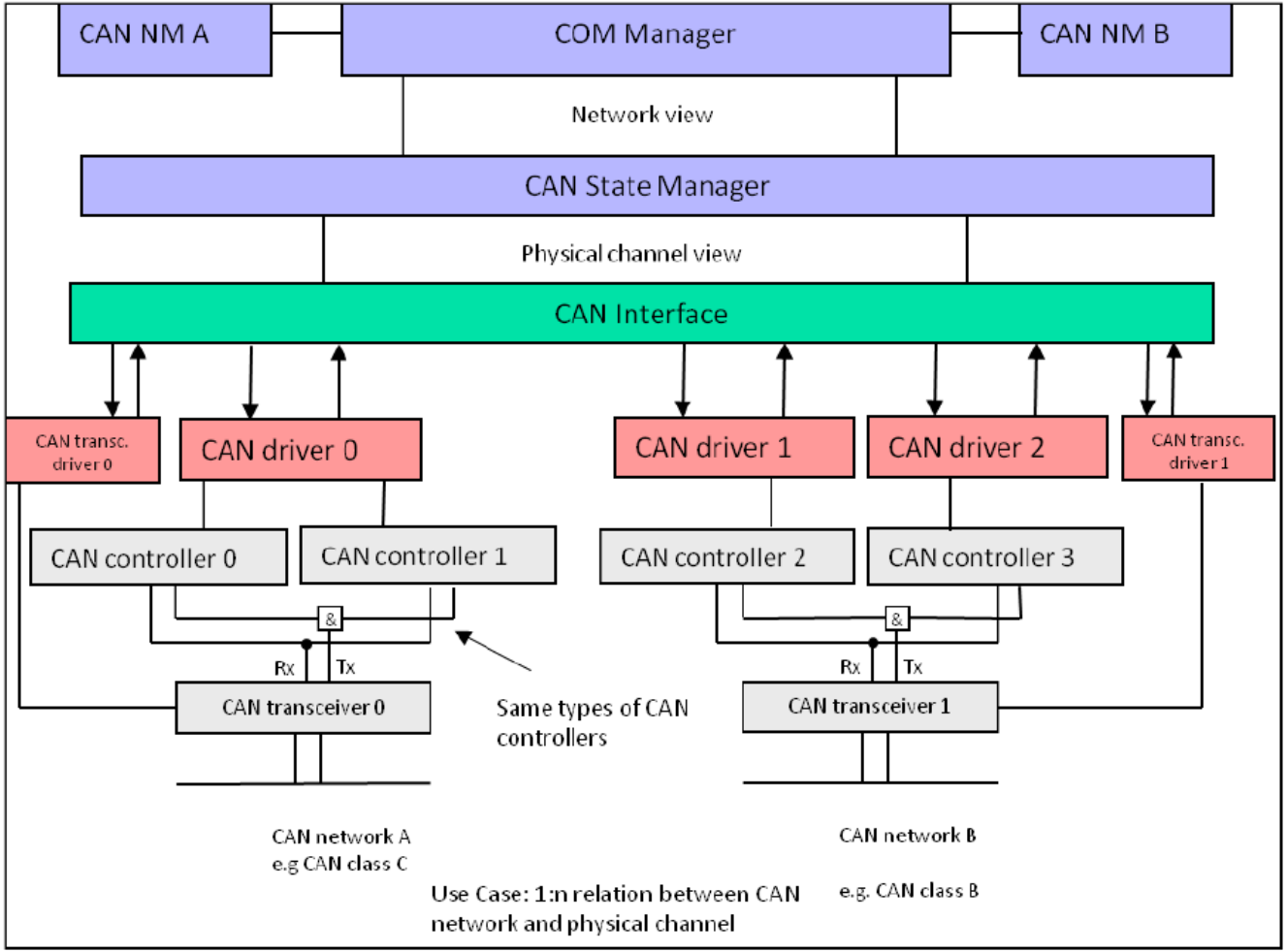
Physical channel view

- A physical channel is linked with one CAN Controller and one CAN Transceiver, whereas one or multiple physical channels may be connected to a single network.
- APIs are used by the CanSm to provide a network view to the ComM used to perform wake up and sleep request for all physical channels connected to a single network.
- The CanIf shall provide a ControllerId and TransceiverId, which abstracts from the different Controllers of the different CanDrv instances and different Transceivers of the different CanTrcv instances respectively

CanIf	CanDrv A	CanDrv B
ControllerId 0	Controller 0	
ControllerId 1	Controller 1	
ControllerId 2		Controller 0

CanIf	CanDrv A	CanDrv B
TransceiverId 0	Transceiver 0	
TransceiverId 1	Transceiver 1	
TransceiverId 2		Transceiver 0


CAN NM A
COM Manager
CAN NM B
CAN State Manager
CAN Interface
CAN transceiver driver 0
CAN driver
CAN Transceiver Driver 1
CAN controller 0
CAN controller 1
Same types of
CAN controllers
CAN transceiver 0
CAN transceiver 1
CAN network A
CAN network B
Only one CAN transceiver driver is needed, of the transceiver type is same.
Different types of CAN transceivers
Use Case: 1:1 relation between CAN network and physical channel


CAN NM A
COM Manager
CAN NM B
Network view
CAN State Manager
Physical channel view
CAN Interface
CAN transc. driver 0
CAN driver 0
CAN driver 1
CAN driver 2
CAN transc. driver 1
CAN controller 0
CAN controller 1
CAN controller 2
CAN controller 3
&
&
Rx
Tx
Rx
Tx
CAN transceiver 0
CAN transceiver 1
Same types of CAN controllers
CAN network A
e.g CAN class C
CAN network B
e.g. CAN class B
Use Case: 1:n relation between CAN network and physical channel

Can Hardware Unit

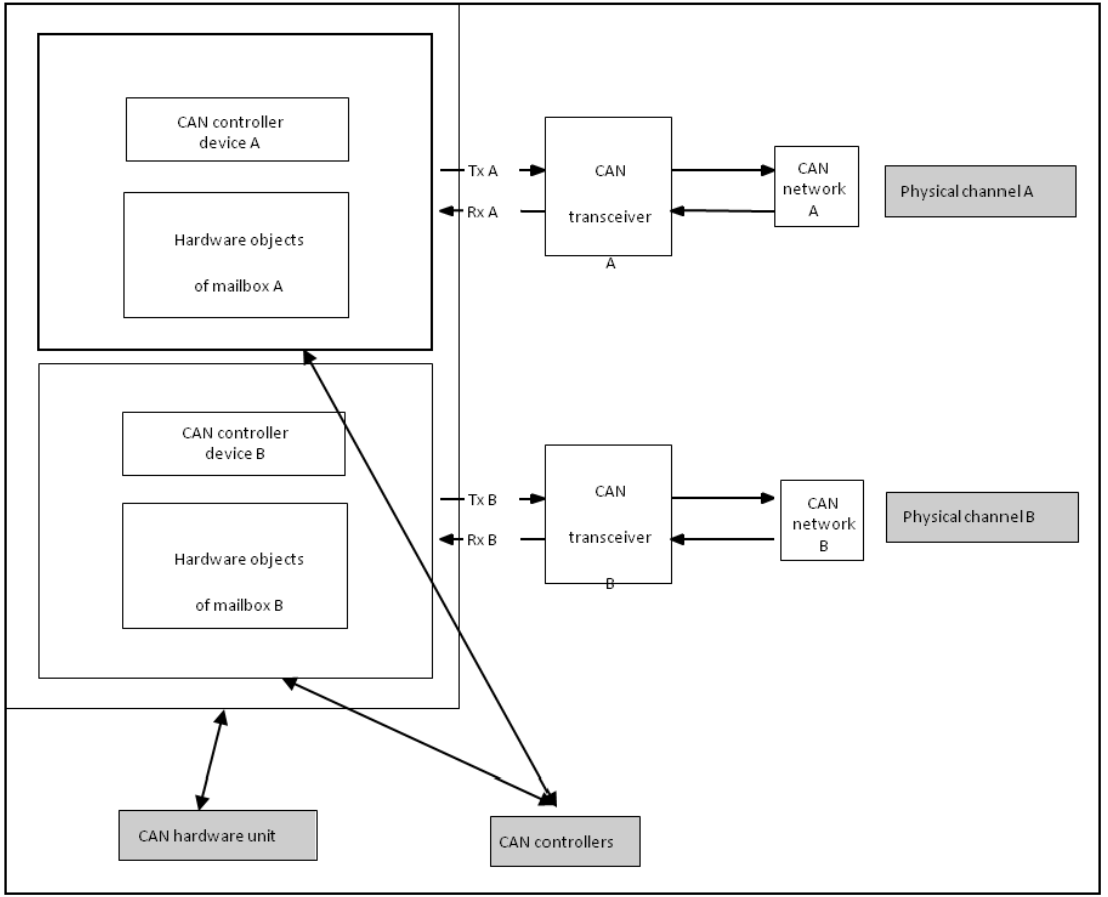


Figure 7.4: Typical CAN Hardware Unit

Basic CAN and Full CAN reception

- CanIf distinguishes between BasicCAN and FullCAN handling for activation of software acceptance filtering.

  - ***FullCAN***: enables transmission or reception of single CanIds.
  - ***BasicCAN***: operation of one HardwareObject enables to transmit or receive a range of CanIds

- CanIf shall execute software filtering to pass only the predefined list of Rx L-PDUs to the corresponding upper layer modules.

- CanIf shall give the possibility to configure and store a software acceptance filter for each HRH of type BasicCAN configured by parameter Can-IfHrhSoftwareFilter.

- CanIf supports reception either of Standard CAN IDs or Extended CAN IDs on one Physical CAN Channel by the parameters CanIfTxPduCanIdType and CanIfRxPduCanIdType.

Initialization

The EcuM calls the CanIf's function CanIf_Init() for initialization of the entire CanIf

- All global variables and data structures are initialized including flags and buffers
- If re-initialization of the entire CAN modules during runtime is required, the EcuM shall invoke the CanSm to initiate the required state transitions of the CAN Controller by call of CAN Interface module's API service CanIf_SetControllerMode

The CanIf maps the calls from CanSm to calls of the respective CanDrvs.

Transmit request

CanIf_Transmit() :

Interface for upper layers to transmit L-PDUs on the CAN network. Transmit Request is successfully completed, if CanDrv could write the L-PDU data into the CAN hardware transmit object.

Can If performs below actions:

- Check, initialization status of CanIf
- Identify CanDrv (only if multiple CanDrvs are used)
- Determine HTH for access to the CAN hardware transmit object
- Call Can_Write() of CanDrv

if CanIf-Transmit() returns E_OK -> Tx is successful

Transmit data flow

- Transmit L-PDU => L-SDU ID,
- Reference to a data structure, to L-SDU related data: Pointer to the L-SDU, pointer to the MetaData and L-SDU length.

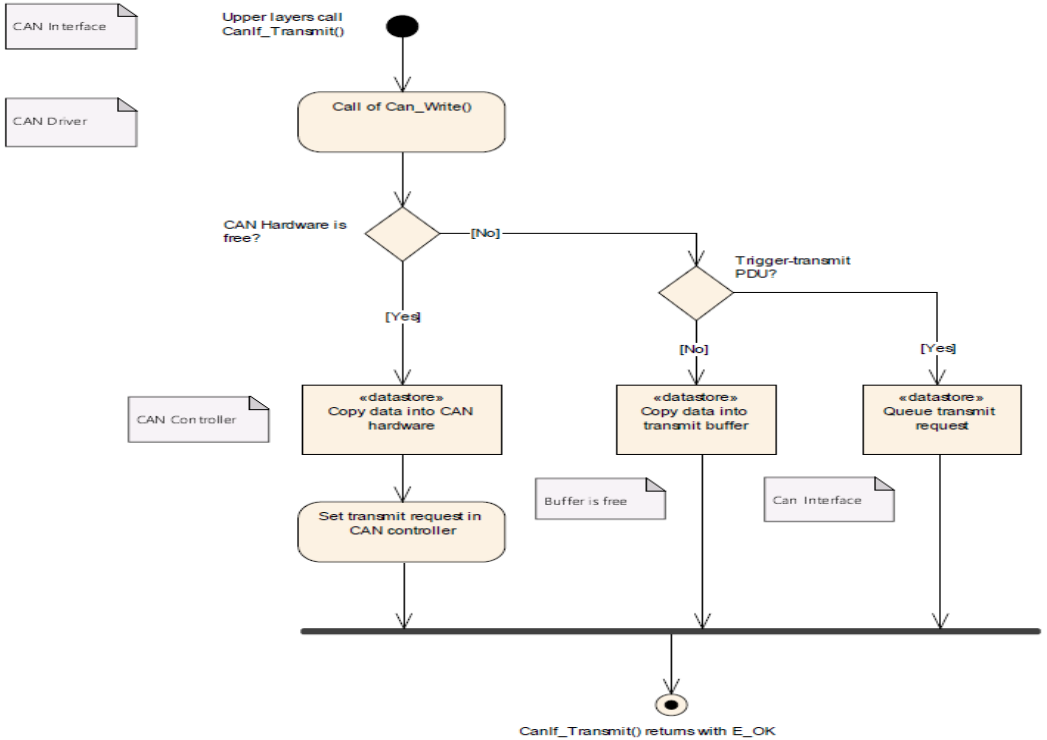


Figure 7.5: Transmit data flow

Transmit buffering

During the transmit process CanIf, CanDrv and the CAN Mailbox altogether shall store the L-PDU to be transmitted only once at a single location. Depending on the transmit method, these are:

- The CAN hardware transmit object or
- The Transmit L-PDU Buffer inside CanIf, if transmit buffering is enabled.

Triggered transmission

- CanIf stores the transmit request for the given L-PDU but not its data. The data is fetched by means of the trigger transmit function when the HTH is free (again).

Transmit buffering

- CanIf will store a Tx L-PDU in a CanIf Transmit L-PDU Buffer (CanIfBufferCfg), if it is rejected by CanDrv at Transmit Request.

- The buffer in CanIf for buffering Tx L-PDUs consists of one or multiple CanIfBufferCfg . Whereas each CanIfBufferCfg is assigned to one or multiple dedicated CanIfBufferHthRef and can be configured to buffer one or multiple Tx L-PDUs. But only one instance per Tx L-PDU can be buffered .

Buffer characteristics

Possible CanIfBufferCfg configurations:

- CanIfTxPduBufferRef
- CanIfBufferCfg
- CanIfBufferHthRef
- CanIf-BufferSize describe

Storage of L-PDUs in the transmit L-PDU buffer

- CanIf tries to store a new Transmit L-PDU or its Transmit Request in the Transmit L-PDU Buffer only, if CanDrv return CAN_BUSY during a call of Can_Write()
- The CanIf shall support buffering of a CAN L-PDU for BasicCAN transmission in the CanIf, if parameter CanIfPublicTxBuffering is enabled.
- For dynamic Transmit L-PDUs, also the CanId has to be stored in the CanIfTxBuffer.
- If the rejected data length exceeds the configured size, CanIf shall: buffer the configured amount of data and discard the rest and report runtime error code CANIF_E_DATA_LENGTH_MISMATCH to the Det_ReportRuntimeError() service of the DET.

Clearance of transmit L-PDU buffers

- CanIf shall transmit L-PDUs or Transmit Requests stored in the Transmit L-PDU Buffers in priority order per each HTH. CanIf shall not differentiate between L-PDUs and Transmit Requests

- When CanIf calls the function Can_Write() for prioritized L-PDUs and Transmit Requests stored in CanIfTxBuffer and the return value of Can_Write() is E_OK, then CanIf shall remove this L-PDU or Transmit Request from the Transmit L-PDU Buffer immediately, before the transmit confirmation returns.

Initialization of transmit L-PDU buffers

- When function CanIf_Init() is called, CanIf shall initialize every Transmit L-PDU Buffer assigned to CanIf.

Data integrity of transmit L-PDU buffers

- CanIf shall protect against concurrent access to Transmit L-PDU Buffers for transmit L-PDUs and Transmit Requests.

- This may be realized by using exclusive areas defined within the BSW Scheduler. These exclusive areas can e.g. configured, that all interrupts will be disabled while the exclusive area is entered. The corresponding services from the BSW Scheduler module are SchM_Enter_CanIf() and SchM_Exit_CanIf().

Transmit confirmation

- If a previous transmit request is completed successfully, CanDrv notifies it to CanIf by the call of CanIf_TxConfirmation()
- When callback notification CanIf_TxConfirmation() is called, CanIf shall identify the upper layer communication layer which is linked to the successfully transmitted L-PDU, and shall notify it about the performed transmission by call of CanIf's transmit confirmation service <User_TxConfirmation>(E_OK).
- One transmit L-PDU can only be assigned to one single transmit confirmation callback service.

Receive data flow

- The received data will be evaluated and processed in the upper layer communication stacks (i.e. AUTOSAR COM, CanNm, CanTp, DCM).

- CanIf provides internal buffering in the receive path only if **CanIfPublicReadRxPduDataApi** is set to TRUE

- In case of a new reception of an L-PDU CanDrv calls CanIf_RxIndication()

The access to the L-PDU specific data is organized by these parameters:

- Hardware Receive Handle (**HRH**)
- Received CAN Identifier (**CanId**)
- Received **Data Length**
- **Reference** to Received L-PDU

- **HRH serves as a link** between CanDrv and the upper layer module using the L-PDU. The HRH identifies one CAN hardware receive object, where a new CAN L-PDU was received.

Receive indication

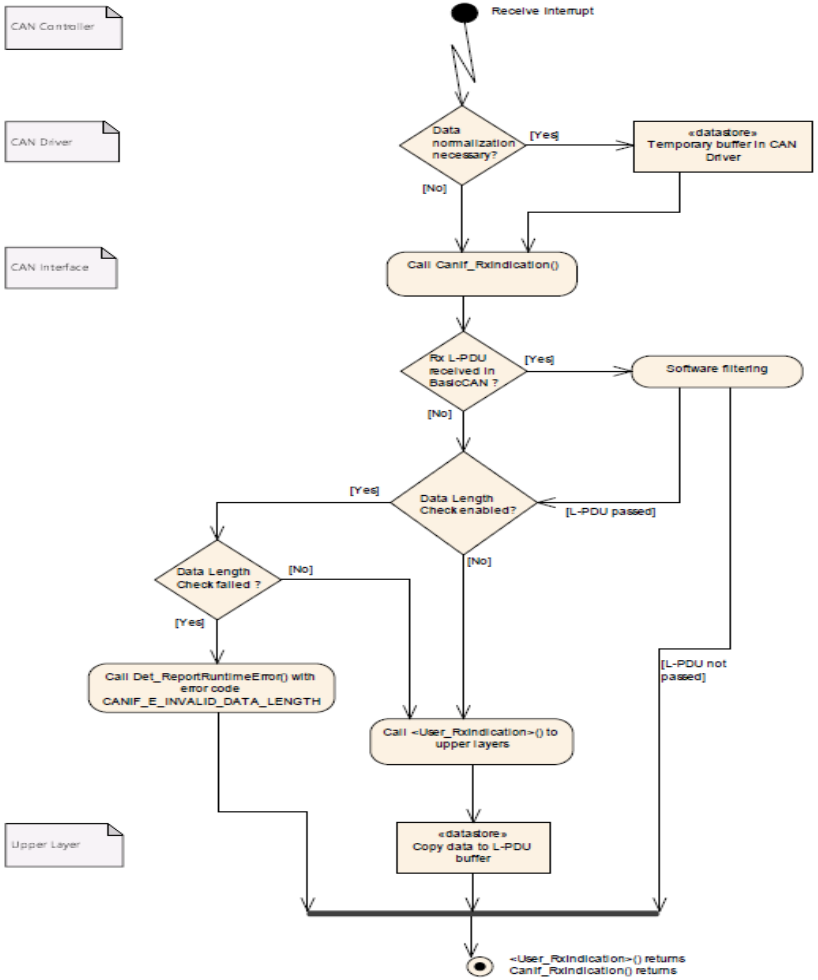
- ***CanIf_RxIndication() :***

If the function CanIf_RxIndication() is called, the CanIf evaluates the CAN L-PDU for acceptance and prepares the L-SDU for later access by the upper communication layers.

The CanIf notifies upper layer modules about this asynchronous event using <User_RxIndication>()

CanIf performs the following steps at a call of CanIf_RxIndication():

- Software Filtering (only BasicCAN), if configured
- Data Length Check, if configured
- buffer received L-SDU if configured
- call upper layer receive indication callback service, if configured.


CAN Controller
CAN Driver
CAN Interface
Upper Layer
Receive Interrupt
Data normalization necessary?
[Yes]
«datastore» Temporary buffer in CAN Driver
[No]
Call CanIf_RxIndication()
Rx L-PDU received in BasicCAN ?
[Yes]
Software filtering
[No]
Data Length Check enabled?
[Yes]
[L-PDU passed]
[No]
Data Length Check failed ?
[No]
[Yes]
Call Det_ReportRuntimeError() with error code CANIF_E_INVALID_DATA_LENGTH
Call <User_RxIndication>() to upper layers
[L-PDU not passed]
«datastore» Copy data to L-PDU buffer
<User_RxIndication>() returns
CanIf_RxIndication() returns

Read received data

- The read received data API **CanIf_ReadRxPduData**() is a common **interface for upper layer** modules to **read CAN L-SDUs** recently received from the CAN network.

- After call of CanIf_RxIndication() and passing of software filtering and Data Length Check, CanIf shall store the received L-SDU in receive L-SDU buffer.

- During the call of CanIf_ReadRxPduData() the assigned receive L-SDU buffer containing a recently received L-SDU, CanIf shall avoid preemptive receive L-SDU buffer access events to that receive L-SDU buffer.

Read Tx/Rx notification status

- CanIf provides the API service CanIf_ReadTxNotifStatus() to read the transmit confirmation status of any transmit L-SDU and the API service CanIf_ReadRxNotifStatus() is provided to read the receive indication status of any receive L-SDU.

- If configuration parameter **CanIfPublicReadTxPduNotifyStatusApi /CanIfPublicReadRxPduNotifyStatusApi** is set to TRUE, CanIf shall store the current notification status for each transmit L-SDU / receive L-SDU.

Data integrity

- **Shared code shall be reentrant** (CanIf shall protect preemptive events, which access shared resources, that could be changed during CanIf's event handling, against each other)

- Handling of **shared transmit and receive L-PDU/L-SDU buffers are critical issues** for the implementation of CanIf. Therefore CanIf shall ensure data integrity and thus use appropriate mechanisms for access to shared **resources like transmission/reception L-PDU/L-SDU buffers**.

- Preemptive events, i.e. transmission and reception event from other CAN Controllers could compromise data integrity by writing into the same L-PDU/L-SDU buffer.

- CanIf can e.g. use CanDrv services to enable (**Can_EnableControllerInterrupts**) and disable (**Can_Disable-ControllerInterrupts**) CAN interrupts and its notifications at **entry and exit of the critical sections** separately for each CAN Controller.

CAN Controller Mode

- The CAN Controller status may be changed at request of the upper layer by the calling of **CanIf_SetControllerMode**() service. The request is passed **by CanIf via the CanDrv API to the addressed CAN Controller.**

- The **consistent management of all CAN Controllers connected at one CAN network is the task of CanSm**. By this way CanSm is responsible to set all CAN Controllers of one CAN network sequentially to sleep mode or to wake them up.

- This network related **state machine is implemented in CanSm**. **CanIf only stores the requested mode and executes the requested transition.**

CAN Controller Operation Modes

- According to the requested operation mode by CanSm, CanIf forwards request Can-Drvs.

Wake-up detection

- The ECU supports wake-up over CAN network, regardless of the used wake-up method (directly about CAN Controller or CAN Transceiver), only if the CAN Controller and CAN Transceiver are set to some kind of "listen for wake-up" mode.

- If wake-up support is enabled CanIf is notified by the Integration Code about a detected CAN wake-up by the service **CanIf_CheckWakeup**()

- In case of a CAN bus "wake-up" event the function **CanIf_CheckWakeup(Wakeup-Source**) may be called during execution of **EcuM_CheckWakeup**(WakeupSource). CanIf in turn checks by configured input reference to **EcuMWakeupSource** in **CanDrvs**, which CanDrvs have to be checked. CanIf gets this information via reference **CanIfCtrlCanCtrlRef**.

- In this way **EcuM** as well as **CanSm** are able **to change CAN Controller States** and to **control the system** behavior concerning **the BusOff recovery or wake-up procedure**.

Wake-up Validation

If **CanIfPublicWakeupCheckValidSupport** equals TRUE -> CanIf enables the detection for CAN wake-up validation.

Therefore, **CanIf stores the event of the first valid call of CanIf_RxIndication() of a CAN Controller** which has been set to **CAN_CS_STARTED**. The first call of CanIf_RxIndication() is valid:

- only for received **NM messages if CanIfPublicWakeupCheckValidByNM is TRUE**
- for all received messages corresponding to configured Rx PDU if CanIfPublicWakeupCheckValidByNM is FALSE.

PDU channel mode control

PDU channel groups

Each L-PDU is assigned to one **dedicated physical CAN channel** connected to one CAN Controller and **one CAN network**. By this way all L-PDUs belonging to one Physical Channel can be controlled on the view **of handling logically single L-PDU channel groups**. Those logical groups represent all L-PDUs of one ECU connected to one underlying CAN network**. PduR & the Network Management** are responsible for controlling the PDU operation modes.

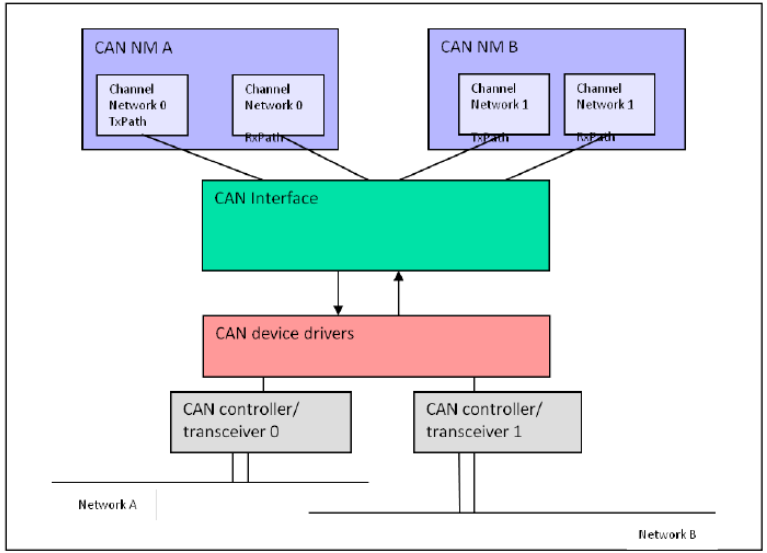


Figure 7.7: Channel PDU groups

PDU channel modes

CanIf provides the services **CanIf_SetPduMode**() and **CanIf_GetPduMode**() to prevent the processing of

- all Transmit L-PDUs belonging to one logical channel
- all Transmit L-PDUs and Receive L-PDUs belonging to one logical channel.

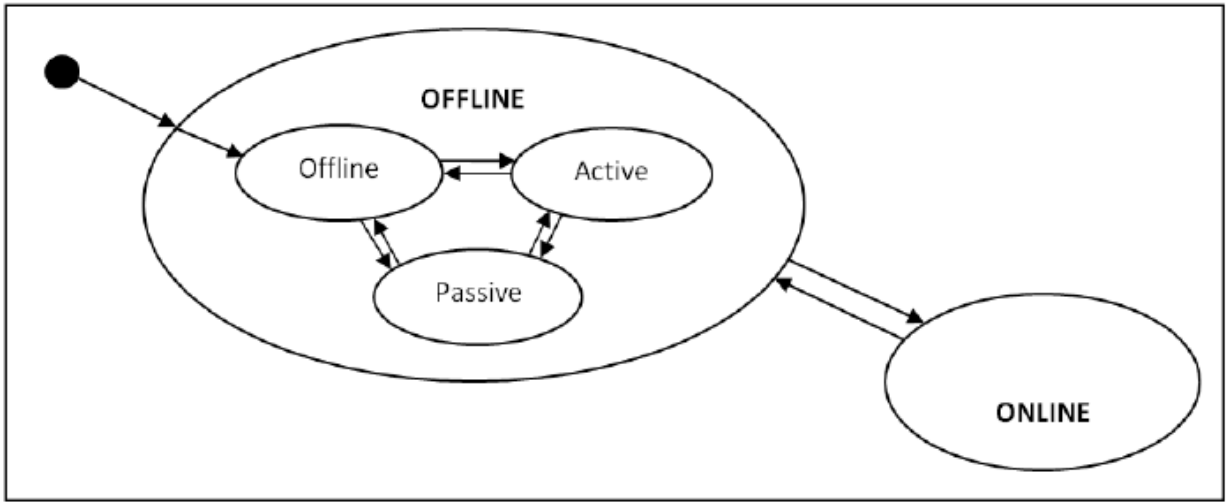


Figure 7.8: PDU channel mode control

CANIF_OFFLINE

During **initialization** CanIf shall switch every channel to **CANIF_OFFLINE**.

CanIf_SetControllerMode(ControllerId, CAN_CS_SLEEP) is called, CanIf shall set the PDU channel mode of the Corresponding channel to CANIF_OFFLINE.

For Physical Channels switching to CANIF_OFFLINE mode CanIf shall:

- prevent forwarding of transmit requests CanIf_Transmit() of associated L- -PDUs to CanDrv
- clear the corresponding CanIf transmit buffers
- prevent invocation of receive indication callback services of the upper layer modules
- prevent invocation of transmit confirmation callback services of the upper layer modules.

CANIF_ONLINE

For Physical Channels switching to CANIF_ONLINE mode CanIf shall:

- enable forwarding of transmit requests CanIf_Transmit() of associated L- -PDUs to CanDrv,
- enable invocation of receive indication callback services of the upper layer modules,
- enable invocation of transmit confirmation callback services of the upper layer modules.

CANIF_OFFLINE_ACTIVE

- If CanIfTxOfflineActiveSupport = TRUE, CanIf provides simulation of successful transmission by CANIF_TX_OFFLINE_ACTIVE mode. This mode is enabled by call of CanIf_SetPduMode(ControllerId, CANIF_TX_OFFLINE_ACTIVE) and only affects the transmission path.

- This functionality is useful to realize special operating modes (i.e. diagnosis passive mode) to avoid bus traffic without impact to the notification mechanism. This mode is typically used for diagnostic usage.

Software receive filter

Certain **software filter algorithms** are provided to optimize software filter runtime. The approach of software filter mechanisms is to find out **the corresponding L-PDU from the HRH and CanId currently being processed**. After the L-PDU is found**, CanIf accepts the reception** and enables upper layers to access L-SDU information directly.

Software filtering concept

The most important settings are the number of the L-PDU hardware objects and their **range**.

- a single Receive L-PDU (FullCAN reception),
- a list of Receive L-PDUs or
- one or multiple ranges of Receive L-PDUs can be linked to a Hardware Receive Object (BasicCAN reception).

- A range of CanIds which shall pass the software receive filter shall either be defined by its upper limit (**CanIfHrhRangeRxPduUpperCanId**) and lower limit (**CanIfHrhRangeRxPduLowerCanId**) CanId, or by a base ID and a mask that defines the relevant bits of the base ID

- Several search algorithms **like linear search, table search and/or hash search variants** to provide the most optimal solution for most use cases.

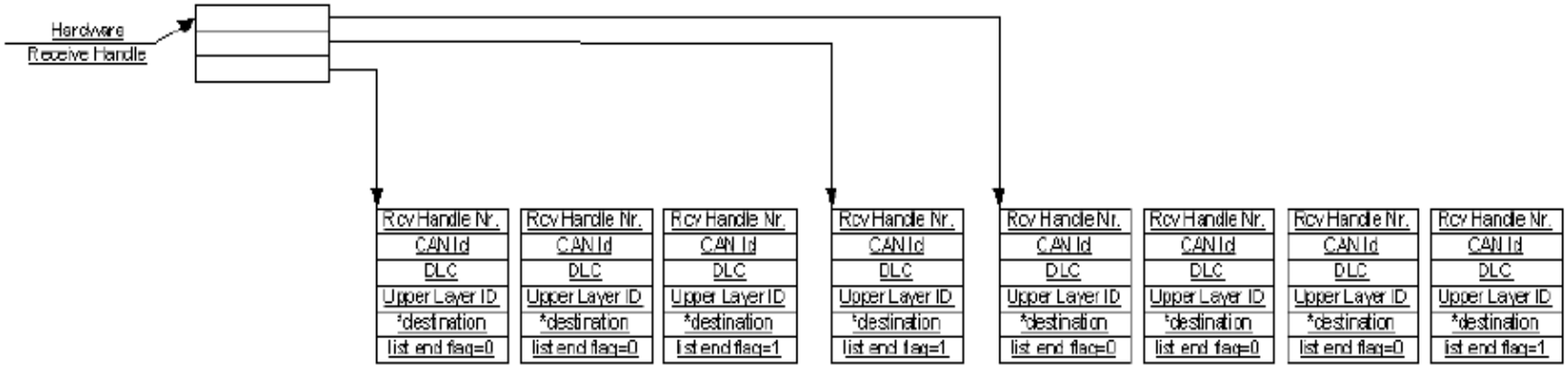


Figure 7.9: Software filtering example

Data Length Check

- The **received Data Length** value is **compared** with **the configured Data Length** value of the **received L-PDU**.
- The configured Data Length value shall be derived from the size of used bytes inside this L-PDU. The configured Data Length value may not be necessarily that Data Length value defined in the CAN communication matrix and used by the sender of this CAN L-PDU.
- CanIf **shall accept all received L-PDUs** with **a Data Length value equal or greater then the configured Data Length value (CanIfRxPduDataLength).**
- If the **Data Length Check rejects a received L-PDU CanIf shall report runtime error code CANIF_E_INVALID_ DATA_LENGTH** to the Det_ReportRuntimeError() service of the DET module.

L-SDU dispatcher to upper layers

- At transmission side the L-SDU dispatcher has to find out the corresponding Tx confirmation callback service of the target upper layer module.
- At reception side each L-SDU belongs to one single upper layer module as destination. This relation is assigned statically at configuration time.
- **The task of the L-SDU dispatcher inside of CanIf is to find out the customer for a received L-SDU and to dispatch the indications towards the found upper layer.**
- These transmit confirmation as well as receive indication notification services may exist several times with different names defined in the notified upper layer modules**. These notification services are statically configured, depending on the layers that have to be served.**

Polling mode

- The polling mode provides **handling of transmit, receive and error** events occurred in the CAN hardware **without the usage of hardware interrupts.**
- Thus the CanIf and the CanDrv provides notification services for detection and execution corresponding hardware events. In polling mode the behavior of these CanIf notification services does not change. By this way upper layer modules are abstracted from the strategy to detect hardware events.
- The CanIf's notification services for forwarding of detected events by the CanDrv are the same like for interrupt operation
- The Polling and Interrupt mode can be configured for each underlying CAN controller.
- If different CanDrvs are in use, the calling frequency has to be harmonized during configuration setup and system integration.

Multiple CAN Driver support

- CanIf needs a specific mapping to cover multiple CanDrv to provide a common interface to upper layers.
- The support for multiple CanDrvs can be enabled and disabled by the configuration parameter **CanIfPublicMultipleDrvSupport**.
- Each CanDrv supports a certain number of underlying CAN Controllers and a fixed number of HTHs/HRHs.
- Each CanDrv has an own numbering area, which starts always at zero for CAN Controllers and HTHs.
- CanIf has to derive the corresponding CanDrv from the L-SDU passed in the APIs. The parameters have to be translated accordingly: i.e. L-SDU => HTH/HRH, CanId, Data Length.

9.1 Transmit request (single CAN Driver)

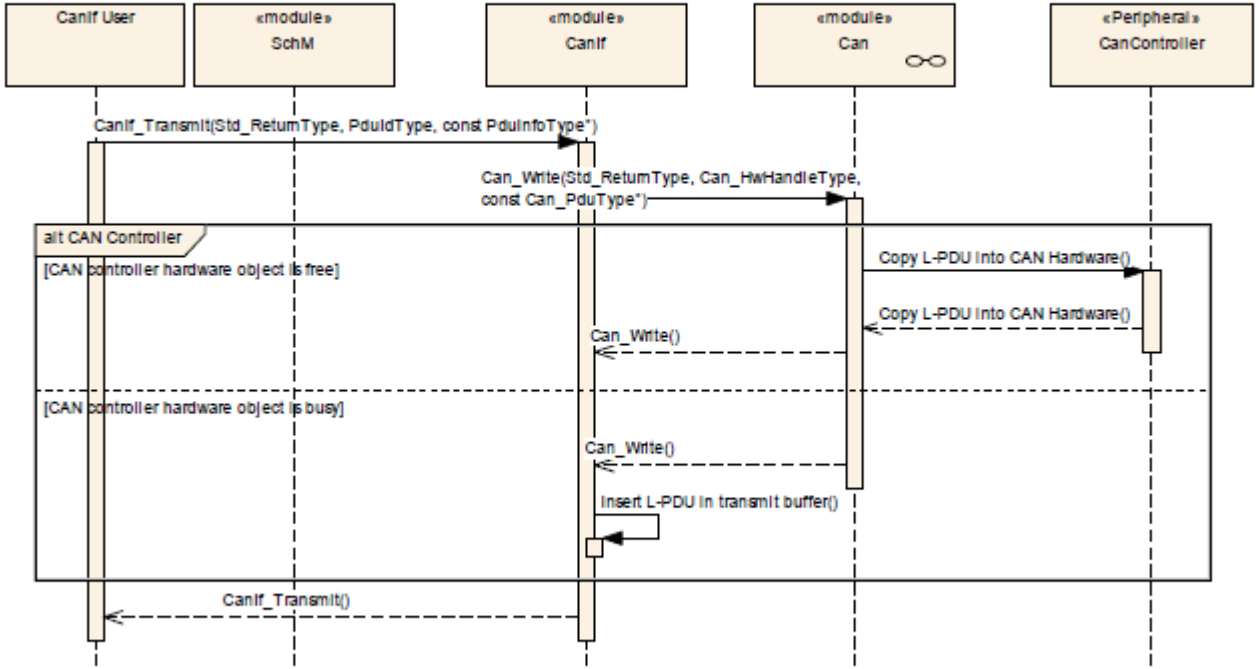


Figure 9.1: Transmission request with a single CAN Driver

Transmit requests by using multiple CAN Drivers

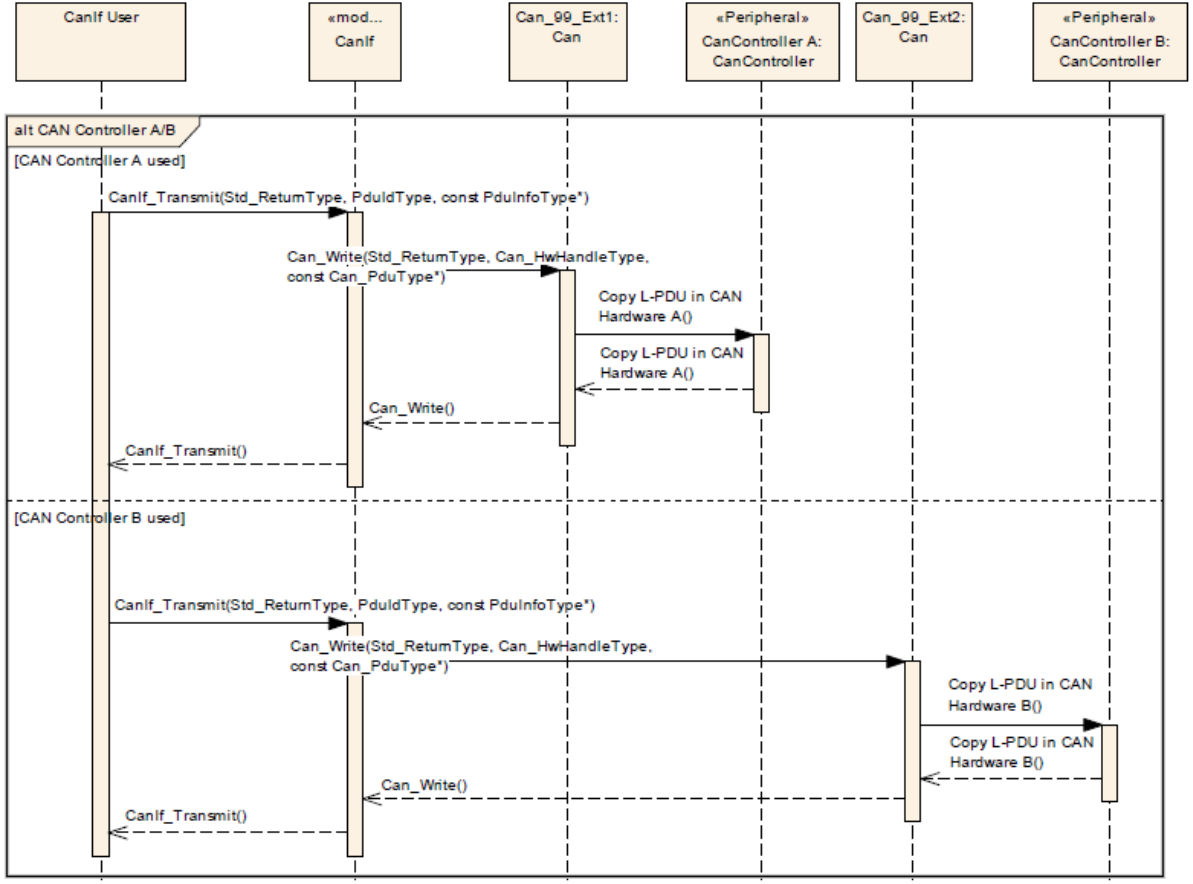
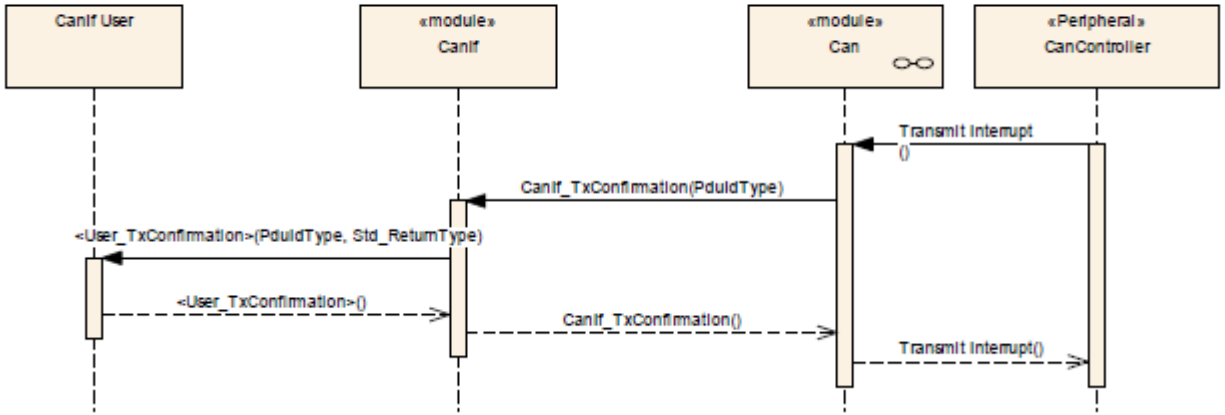
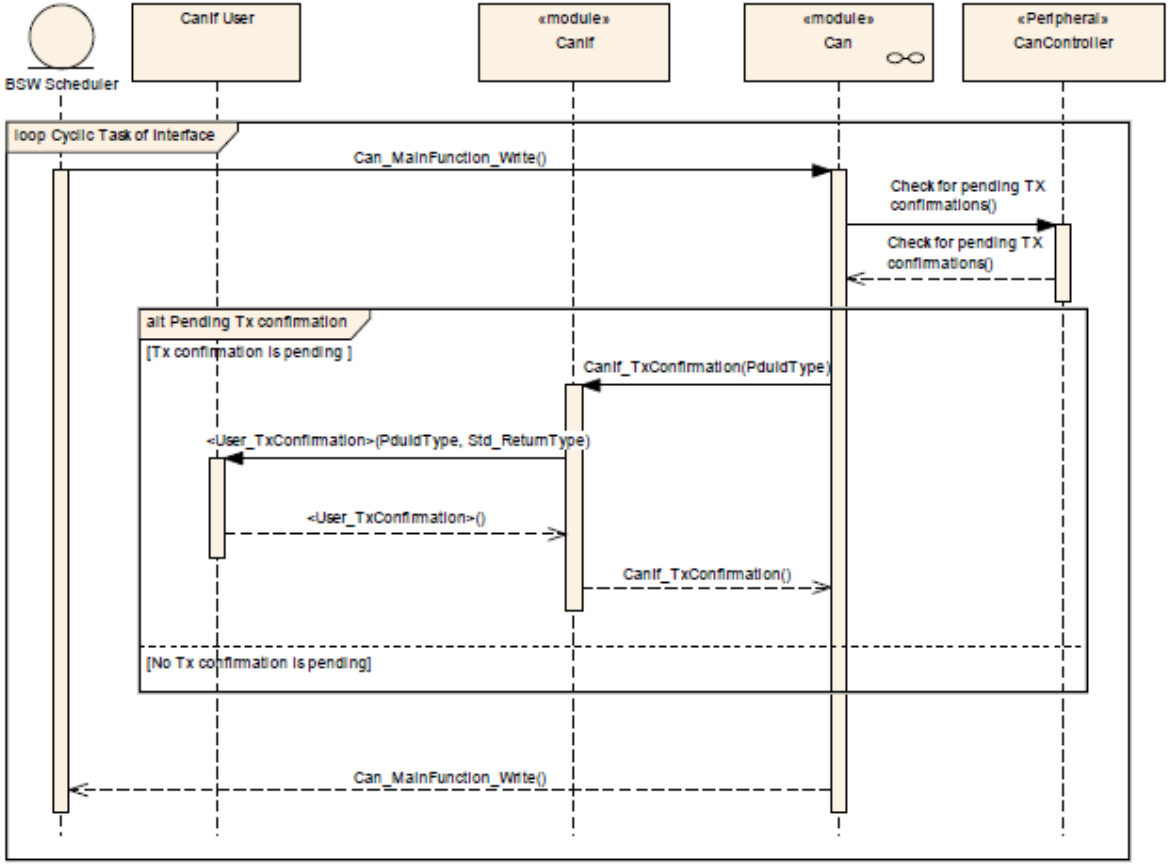


Figure 7.10: Transmission request with multiple CAN Drivers - simplified

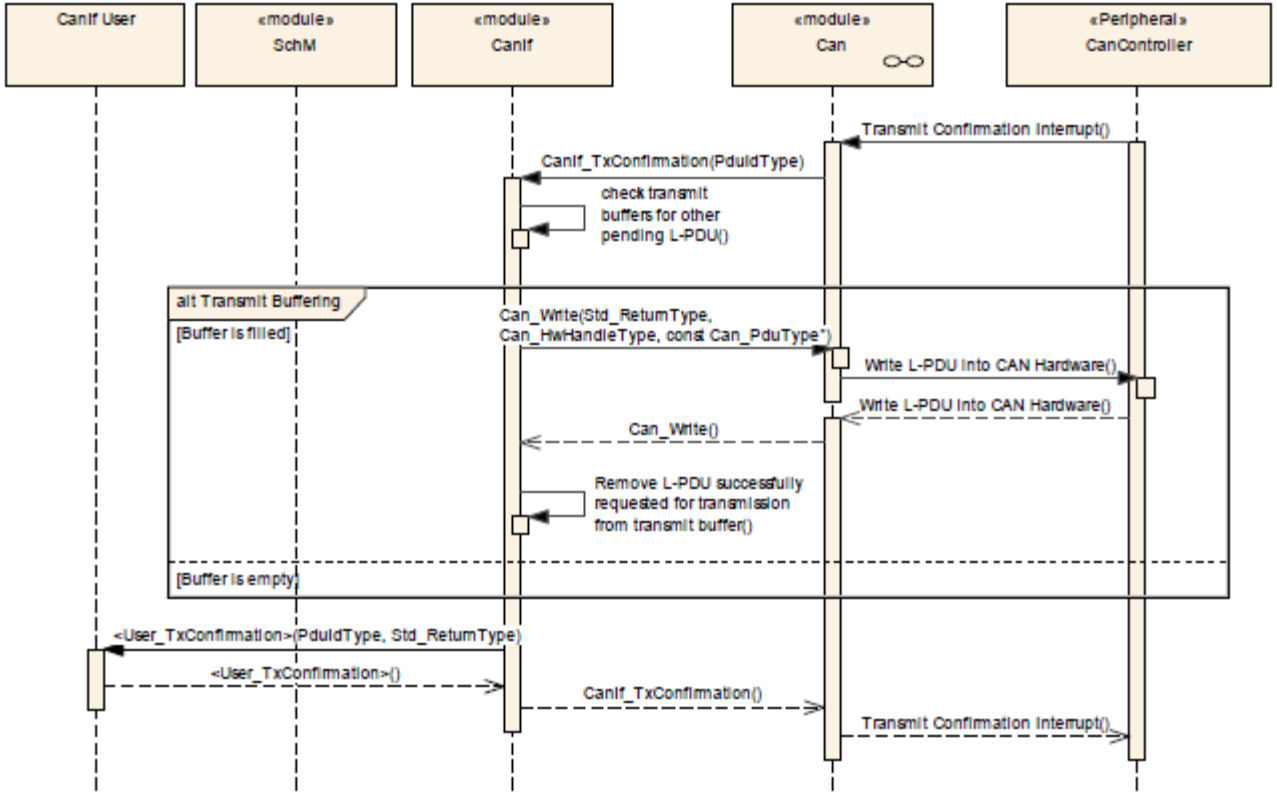
9.3 Transmit confirmation (interrupt mode)



9.4 Transmit confirmation (polling mode)



9.5 Transmit confirmation (with buffering)



9.6 Trigger Transmit Request

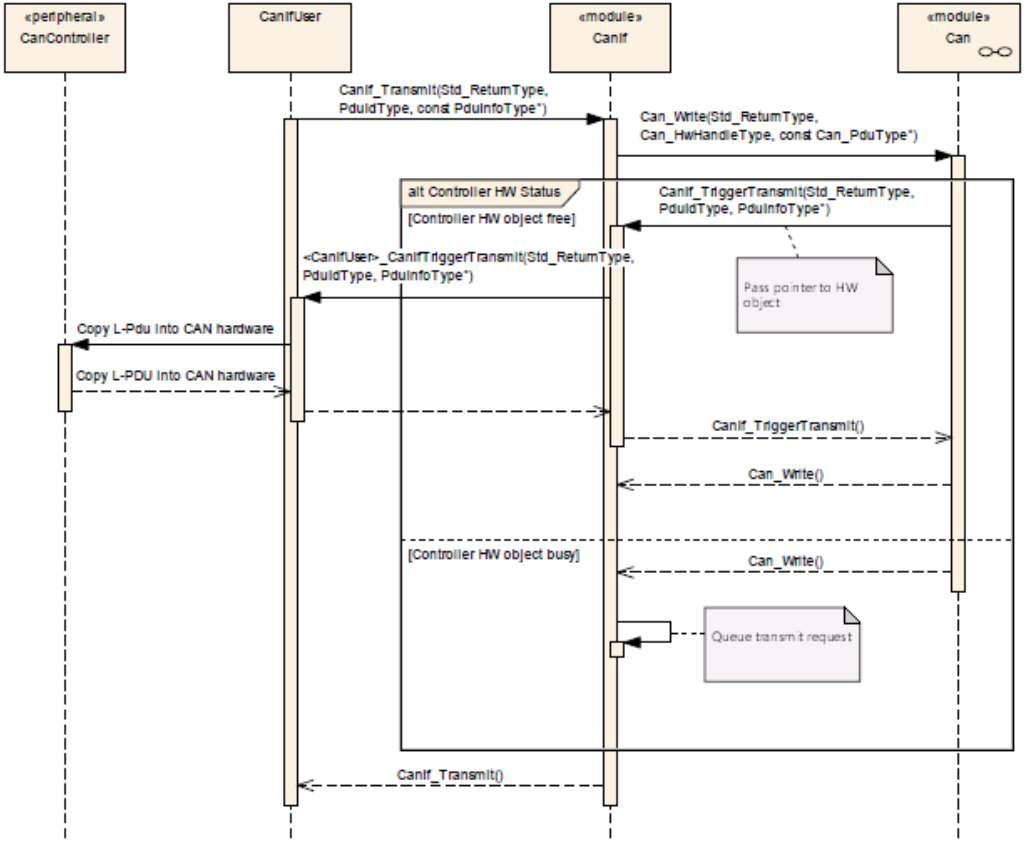


Figure 9.6: Trigger Transmit Request

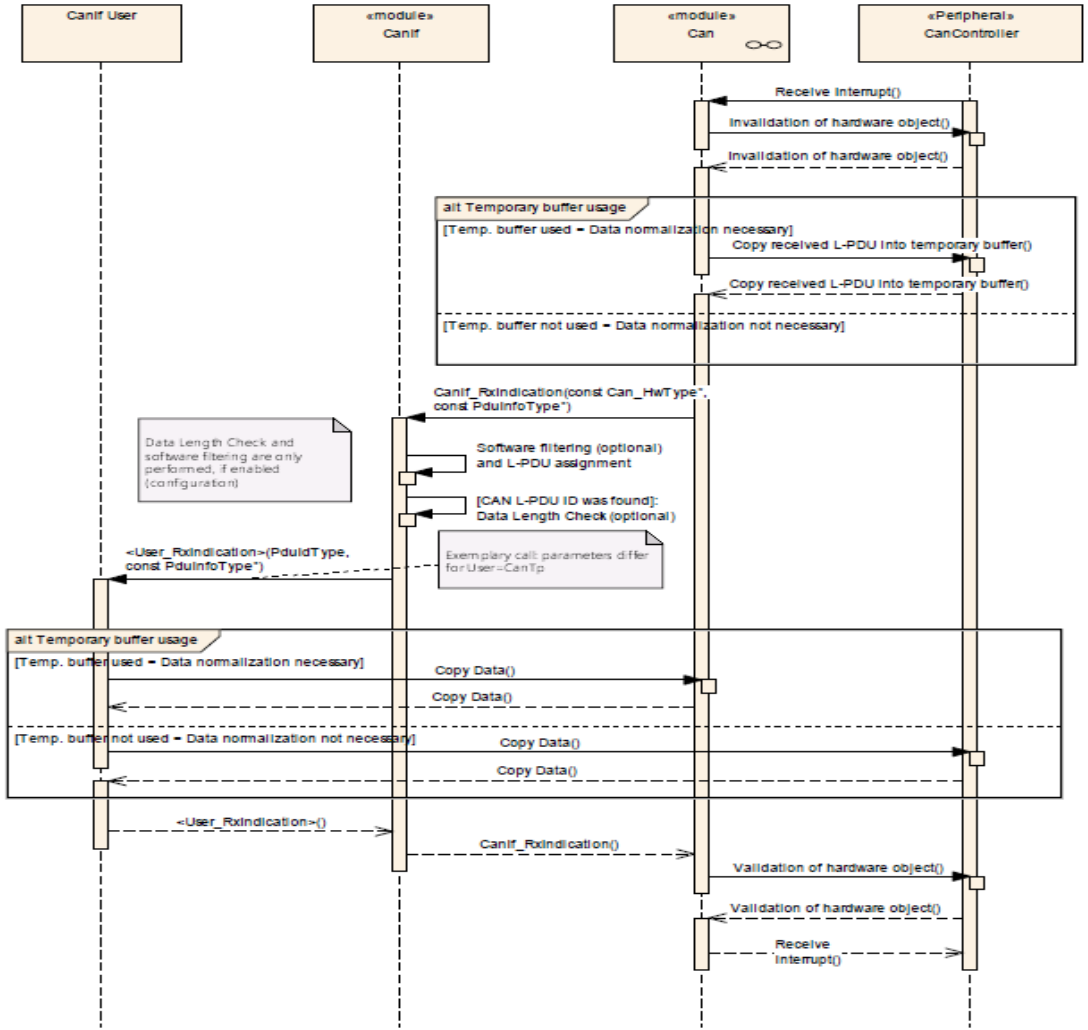
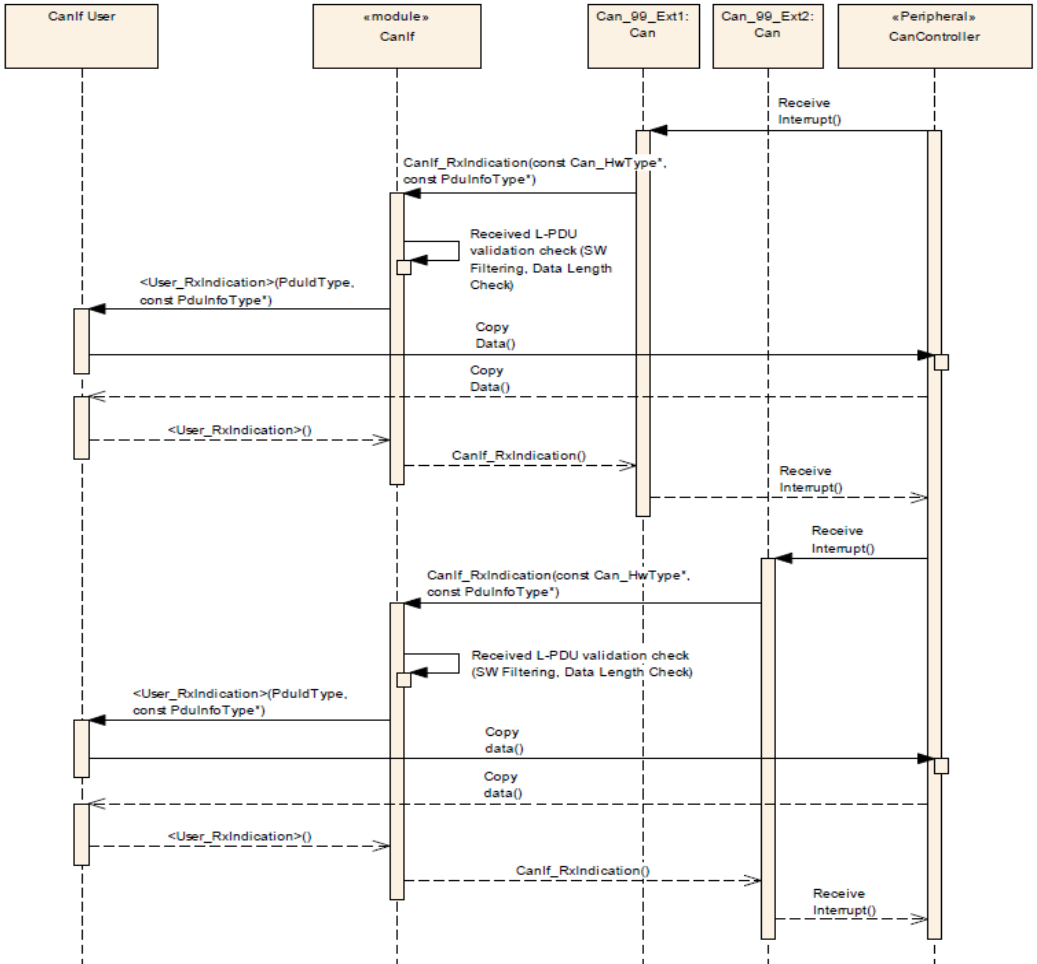


Figure 9.7: Receive indication interrupt driven

Notification mechanism using multiple CAN Drivers



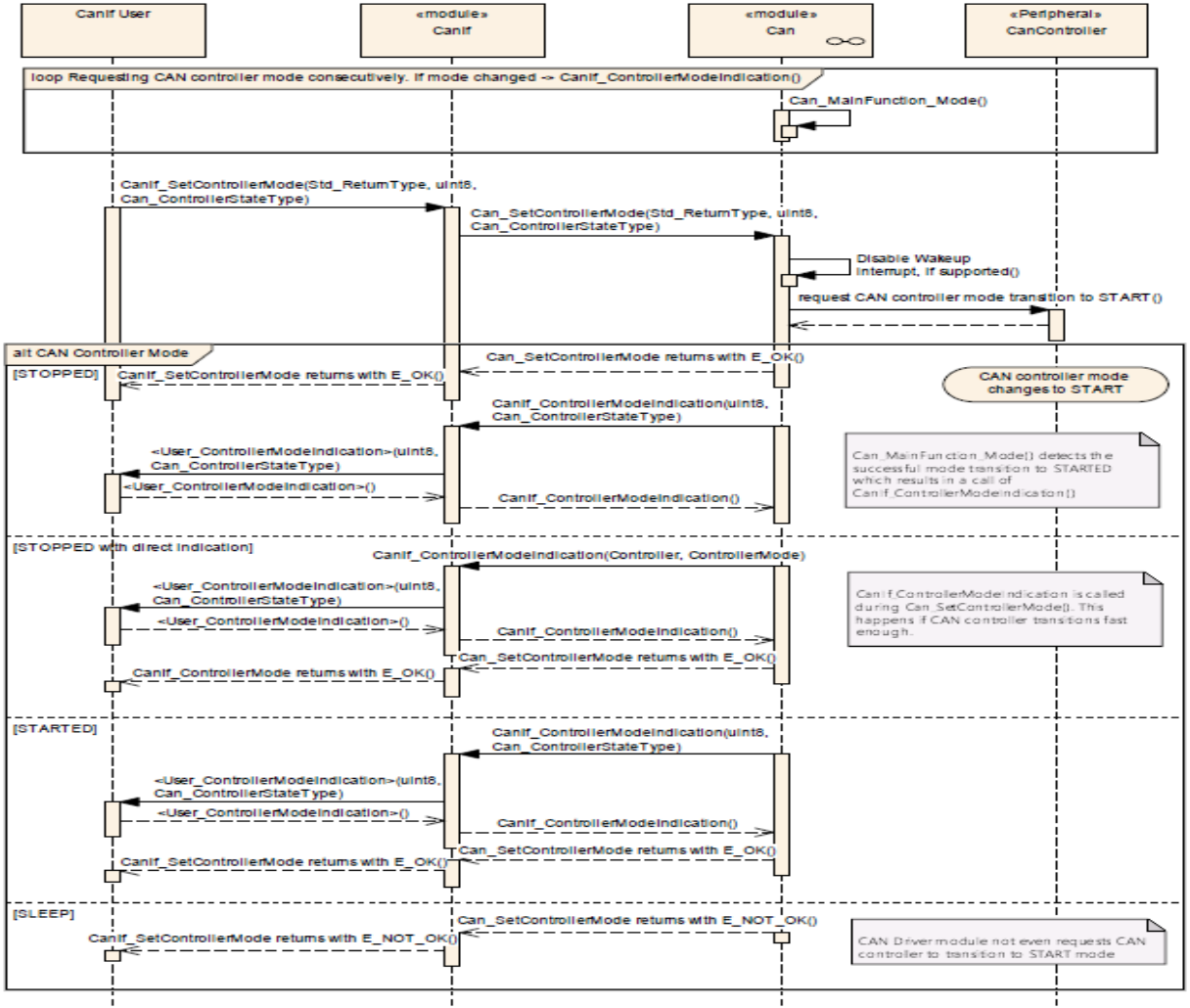


Figure 9.10: Start CAN network

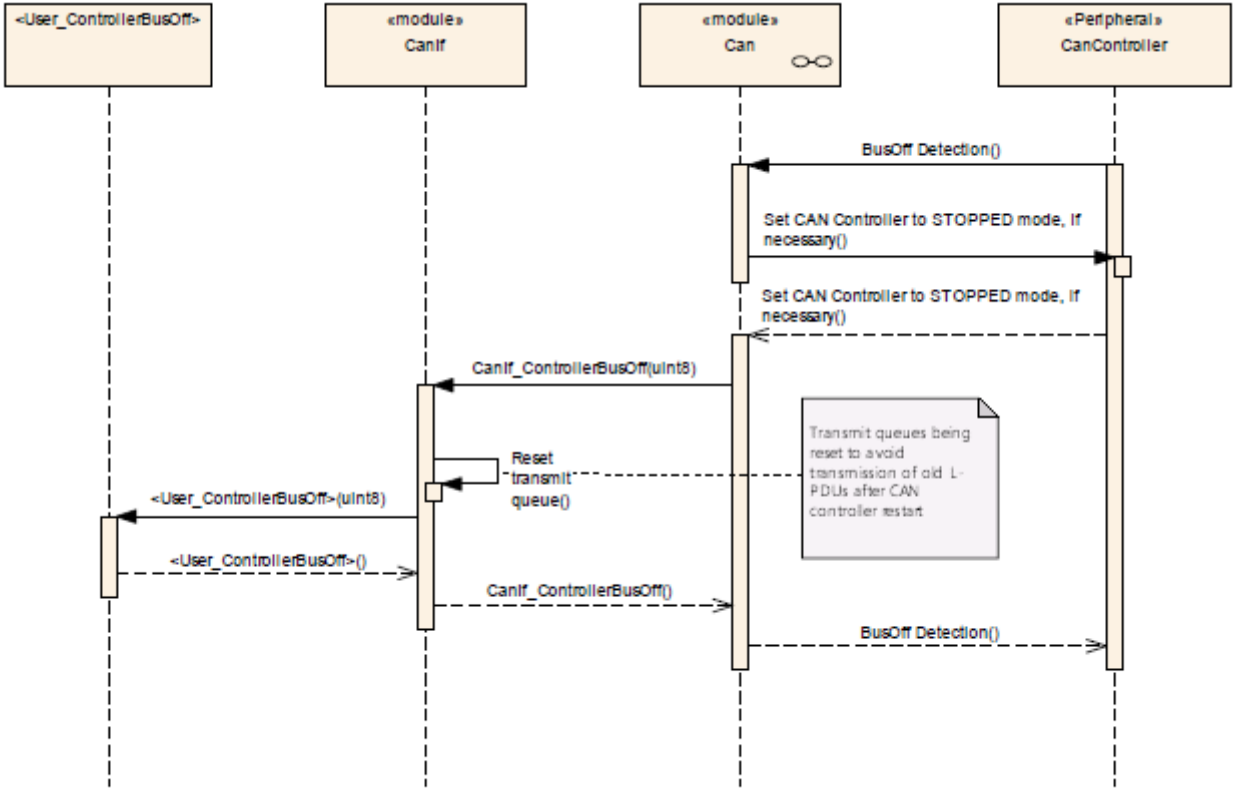


Figure 9.11: BusOff notification

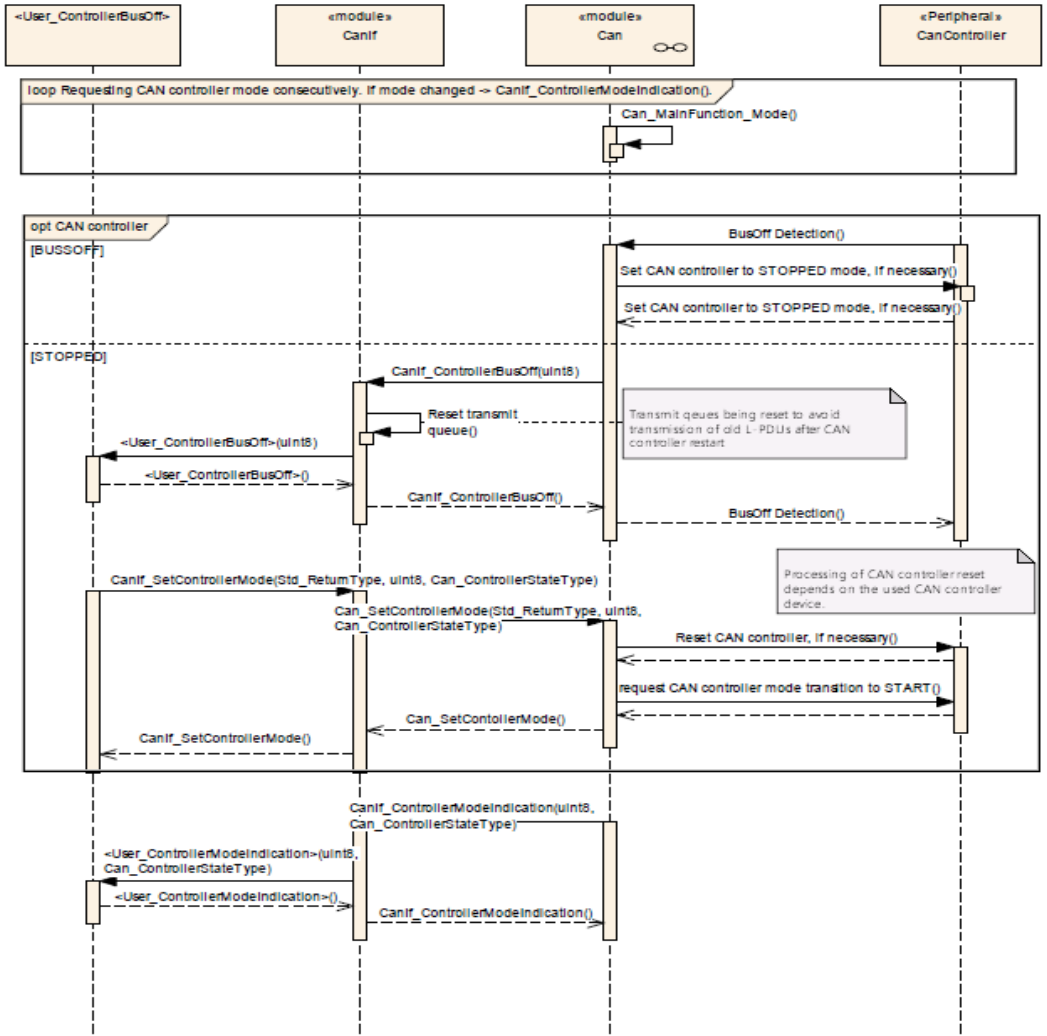


Figure 9.12: BusOff recovery

Partial Networking

- If Partial Networking (PN) is enabled (**CanIfPublicPnSupport**), CanIf shall support a **PnTxFilter per CAN Controller**
- The **PnTxFilter** of shall only have an effect and transition its modes (enabled/disabled) if more than zero Tx L-PDUs per CAN Controller are configured as **CanIfTxPduPnFilterPdu**.
- **PnTxFilter** shall be **enabled during initialization**
- If CanIf_SetControllerMode(ControllerId, CAN_CS_SLEEP) is called the PnTxFilter of the corresponding CAN Controller shall be enabled
- If the PnTxFilter of a CAN Controller is enabled, **CanIf shall block all Tx requests to that CAN Controller** (return E_NOT_OK when CanIf_Transmit() is called), **except if the requested Tx L-PDUs** is one of the configured **CanIfTxPduPnFilterPdus** of that CAN Controller.

CAN Fd Support

- For performance reasons some CAN Controllers allow to use a Flexible Data-Rate feature called CAN FD
- Besides, the higher baud rate for the payload CAN FD also supports an extended payload which allows the transmission of up to 64 bytes.
- If an L-SDU shall be sent as CAN FD or conventional CAN 2.0 frame depends on the configured **CanIfTxPduCanIdType**. CanIf indicates this to CanDrv utilizing the **second most significant bit of PduInfo->id** (**Can_IdType**) passed while calling **Can_Write**().
- Which kind of frame was received by CanDrv is also indicated utilizing the **second most significant bit** of the **Can_IdType** passed with **CanIf_RxIndication**() (Mailbox-->CanId). Based on this information CanIf decides how to map to the configured L-SDU (**CanIfRxPduCfg**)

Security Events

If security event reporting has been enabled for the CanIf module (**CanIfEnableSecurityEventReporting** = true) the respective security events shall bereported to the IdsM via the defined interfaces

Name	*Description*	*ID*
CANIF_SEV_TX_ERROR_DETECTED	A transmission related error was detected. Depending on the context data this could indicate suspicious CAN activity.	19
CANIF_SEV_RX_ERROR_DETECTED	A reception related error was detected. Depending on the context data this could indicate suspicious CAN activity.	20
CANIF_SEV_ERRORSTATE_PASSIVE	The CAN controller transitioned to state passive.	21
CANIF_SEV_ERRORSTATE_BUSOFF	The CAN controller transitioned to state busoff.	22

Development Errors

Type of error	*Related error code*	*Error value*
API service called with invalid CAN ID	CANIF_E_PARAM_CANID	10
API service called with invalid hardware object	CANIF_E_PARAM_HOH	12
API service called with invalid PDU ID	CANIF_E_PARAM_LPDU	13
API service called with invalid controller ID	CANIF_E_PARAM_CONTROLLERID	15
API service called with invalid wakeup source	CANIF_E_PARAM_WAKEUPSOURCE	16
API service called with invalid transceiver ID	CANIF_E_PARAM_TRCV	17
API service called with invalid transceiver mode	CANIF_E_PARAM_TRCVMODE	18
API service called with invalid transceiver wakeup mode	CANIF_E_PARAM_TRCVWAKEUPMODE	19
API service called with invalid pointer	CANIF_E_PARAM_POINTER	20
API service called with invalid controller mode	CANIF_E_PARAM_CTRLMODE	21
API service called with invalid PDU mode	CANIF_E_PARAM_PDU_MODE	22
API services called with invalid parameter	CANIF_E_PARAM_CAN_ERROR	23
API service used without module initialization	CANIF_E_UNINIT	30
Transmit PDU ID invalid	CANIF_E_INVALID_TXPDUID	50
Receive PDU ID invalid	CANIF_E_INVALID_RXPDUID	60
CAN Interface initialisation failed	CANIF_E_INIT_FAILED	80

Runtime Errors

Type of error	*Related error code*	*Error value*
Failed Data Length Check	CANIF_E_INVALID_DATA_LENGTH	61
Data Length	CANIF_E_DATA_LENGTH_MISMATCH	62
Transmit requested on offline PDU channel	CANIF_E_STOPPED	70
Message length was exceeding the maximum length	CANIF_E_TXPDU_LENGTH_EXCEEDED	90

CAN If Api's

CanIf_Init	void CanIf_Init **(**const CanIf_ConfigType***** ConfigPtr**)**
CanIf_DeInit	void CanIf_DeInit **(**void**)**
CanIf_SetControllerMode	Std_ReturnType CanIf_SetControllerMode **(**uint8 ControllerId, Can_ControllerStateType ControllerMode**)**
CanIf_GetControllerMode	Std_ReturnType CanIf_GetControllerMode **(**uint8 ControllerId, Can_ControllerStateType***** ControllerModePtr**)**
CanIf_GetControllerErrorState	Std_ReturnType CanIf_GetControllerErrorState **(**uint8 ControllerId, Can_ErrorStateType***** ErrorStatePtr **)**
CanIf_Transmit	Std_ReturnType CanIf_Transmit **(** PduIdType TxPduId, const PduInfoType***** PduInfoPtr **)**
CanIf_ReadRxPduData	Std_ReturnType CanIf_ReadRxPduData **(** PduIdType CanIfRxSduId, PduInfoType***** CanIfRxInfoPtr **)**
CanIf_ReadTxNotifStatus	CanIf_NotifStatusType CanIf_ReadTxNotifStatus **(** PduIdType CanIfTxSduId**)**
CanIf_ReadRxNotifStatus	CanIf_NotifStatusType CanIf_ReadRxNotifStatus **(** PduIdType CanIfRxSduId**)**
CanIf_SetPduMode	Std_ReturnType CanIf_SetPduMode **(** uint8 ControllerId, CanIf_PduModeType PduModeRequest**)**

CanIf_GetPduMode	Std_ReturnType CanIf_GetPduMode **(** uint8 ControllerId, CanIf_PduModeType*** PduModePtr)**
CanIf_GetVersionInfo	void CanIf_GetVersionInfo **(** Std_VersionInfoType*** VersionInfo)**
CanIf_SetDynamicTxId	void CanIf_SetDynamicTxId **(** PduIdType CanIfTxSduId,Can_IdType CanId **)**
CanIf_SetTrcvMode	Std_ReturnType CanIf_SetTrcvMode **(** uint8 TransceiverId, CanTrcv_TrcvModeType TransceiverMode**)**
CanIf_GetTrcvMode	Std_ReturnType CanIf_GetTrcvMode **(** uint8 TransceiverId, CanTrcv_TrcvModeType***** TransceiverModePtr **)**
CanIf_GetTrcvWakeupReason	Std_ReturnType CanIf_GetTrcvWakeupReason **(** uint8 TransceiverId, CanTrcv_TrcvWakeupReasonType***** TrcvWuReasonPtr **)**
CanIf_SetTrcvWakeupMode	Std_ReturnType CanIf_SetTrcvWakeupMode **(** uint8 TransceiverId, CanTrcv_TrcvWakeupModeType TrcvWakeupMode**)**
CanIf_CheckWakeup	Std_ReturnType CanIf_CheckWakeup **(** EcuM_WakeupSourceType WakeupSource **)**
CanIf_CheckValidation	Std_ReturnType CanIf_CheckValidation **(** EcuM_WakeupSourceType WakeupSource **)**
CanIf_GetTxConfirmationState	CanIf_NotifStatusType CanIf_GetTxConfirmationState **(**uint8 ControllerId**)**

CanIf_ClearTrcvWufFlag	Std_ReturnType CanIf_ClearTrcvWufFlag **(**uint8 TransceiverId**)**
CanIf_CheckTrcvWakeFlag	Std_ReturnType CanIf_CheckTrcvWakeFlag **(**uint8 TransceiverId**)**
CanIf_SetBaudrate	Std_ReturnType CanIf_SetBaudrate **(**uint8 ControllerId, uint16 BaudRateConfigID**)**
CanIf_GetControllerRxErrorCounter CanIf_GetControllerTxErrorCounter	CanIf_GetControllerRxErrorCounter **(**uint8 ControllerId, uint8***** RxErrorCounterPtr**)** CanIf_GetControllerTxErrorCounter **(**uint8 ControllerId, uint8***** TxErrorCounterPtr **)**
CanIf_EnableBusMirroring	Std_ReturnType CanIf_EnableBusMirroring **(**uint8 ControllerId, boolean MirroringActive**)**
CanIf_GetCurrentTime	Std_ReturnType CanIf_GetCurrentTime **(**uint8 Controller, Can_TimeStampType***** timeStampPtr**)**
CanIf_EnableEgressTimeStamp	void CanIf_EnableEgressTimeStamp **(**PduIdType TxPduId**)**
CanIf_GetEgressTimeStamp	Std_ReturnType CanIf_GetEgressTimeStamp **(**PduIdType TxPduId, Can_TimeStampType***** timeStampPtr**)**
CanIf_GetIngressTimeStamp	Std_ReturnType CanIf_GetIngressTimeStamp **(**PduIdType RxPduId, Can_TimeStampType***** timeStampPtr**)**

Callback notifications

CanIf_TriggerTransmit	Std_ReturnType CanIf_TriggerTransmit **(** PduIdType TxPduId, PduInfoType***** PduInfoPtr **)**
CanIf_TxConfirmation	void CanIf_TxConfirmation **(** PduIdType CanTxPduId **)**
CanIf_RxIndication	void CanIf_RxIndication **(** const Can_HwType***** Mailbox**,** const PduInfoType***** PduInfoPtr **)**
CanIf_ControllerBusOff	void CanIf_ControllerBusOff **(** uint8 ControllerId **)**
CanIf_ConfirmPnAvailability	void CanIf_ConfirmPnAvailability **(** uint8 TransceiverId **)**
CanIf_ClearTrcvWufFlagIndication	void CanIf_ClearTrcvWufFlagIndication **(** uint8 TransceiverId **)**
CanIf_CheckTrcvWakeFlagIndication	void CanIf_CheckTrcvWakeFlagIndication **(** uint8 TransceiverId **)**
CanIf_ControllerModeIndication	void CanIf_ControllerModeIndication **(** uint8 ControllerId, Can_ControllerStateType ControllerMode **)**
CanIf_TrcvModeIndication	void CanIf_TrcvModeIndication **(** uint8 TransceiverId, CanTrcv_TrcvModeType TransceiverMode **)**
CanIf_ControllerErrorStatePassive	void CanIf_ControllerErrorStatePassive **(** uint8 ControllerId, uint16 RxErrorCounter, uint16 TxErrorCounter **)**
CanIf_ErrorNotification	void CanIf_ErrorNotification **(** uint8 ControllerId, Can_ErrorType Can_ErrorType**)**

Scheduled functions

Note: CanIf does not have scheduled functions or needs some.

Mandatory interfaces

API Function	Header File	Description
Can_GetControllerErrorState	Can.h	This service obtains the error state of the CAN controller.
Can_GetControllerRxErrorCounter	Can.h	Returns the Rx error counter for a CAN controller. This value might not be available for all CAN controllers, in which case E_NOT_OK would be returned. Please note that the value of the counter might not be correct at the moment the API returns it, because the Rx counter is handled asynchronously in hardware. Applications should not trust this value for any assumption about the current bus state.
Can_GetControllerTxErrorCounter	Can.h	Returns the Tx error counter for a CAN controller. This value might not be available for all CAN controllers, in which case E_NOT_OK would be returned. Please note that the value of the counter might not be correct at the moment the API returns it, because the Tx counter is handled asynchronously in hardware. Applications should not trust this value for any assumption about the current bus state.
Can_SetControllerMode	Can.h	This function performs software triggered state transitions of the CAN controller State machine.
Can_Write	Can.h	This function is called by CanIf to pass a CAN message to CanDrv for transmission.
Det_ReportRuntimeError	Det.h	Service to report runtime errors. If a callout has been configured then this callout shall be called.
SchM_Enter_CanIf_<ExclusiveArea>	SchM_<Mip>.h	Invokes the SchM_Enter function to enter a module local exclusive area.
SchM_Exit_CanIf_<ExclusiveArea>	SchM_<Mip>.h	Invokes the SchM_Exit function to exit an exclusive area.

Configurable interfaces

<User_TriggerTransmit>	Std_ReturnType <User_TriggerTransmit> **(** PduIdType TxPduId, PduInfoType***** PduInfoPtr **)**
<User_TxConfirmation>	void <User_TxConfirmation> **(** PduIdType TxPduId, Std_ReturnType result **)**
<User_RxIndication>	void <User_RxIndication> **(** PduIdType RxPduId, const PduInfoType***** PduInfoPtr **)**
<User_ValidateWakeupEvent>	void <User_ValidateWakeupEvent> **(** EcuM_WakeupSourceType sources **)**
<User_ControllerBusOff>	void <User_ControllerBusOff> **(** uint8 ControllerId **)**
<User_ConfirmPnAvailability>	void <User_ConfirmPnAvailability> **(** uint8 TransceiverId **)**
<User_ClearTrcvWufFlagIndication>	void <User_ClearTrcvWufFlagIndication> **(** uint8 TransceiverId **)**
<User_CheckTrcvWakeFlagIndication>	void <User_CheckTrcvWakeFlagIndication> **(** uint8 TransceiverId **)**
<User_ControllerModeIndication>	void <User_ControllerModeIndication> **(** uint8 ControllerId, Can_ControllerStateType ControllerMode **)**
<User_TrcvModeIndication>	void <User_TrcvModeIndication> **(** uint8 TransceiverId, CanTrcv_TrcvModeType TransceiverMode **)**

Sequence diagrams

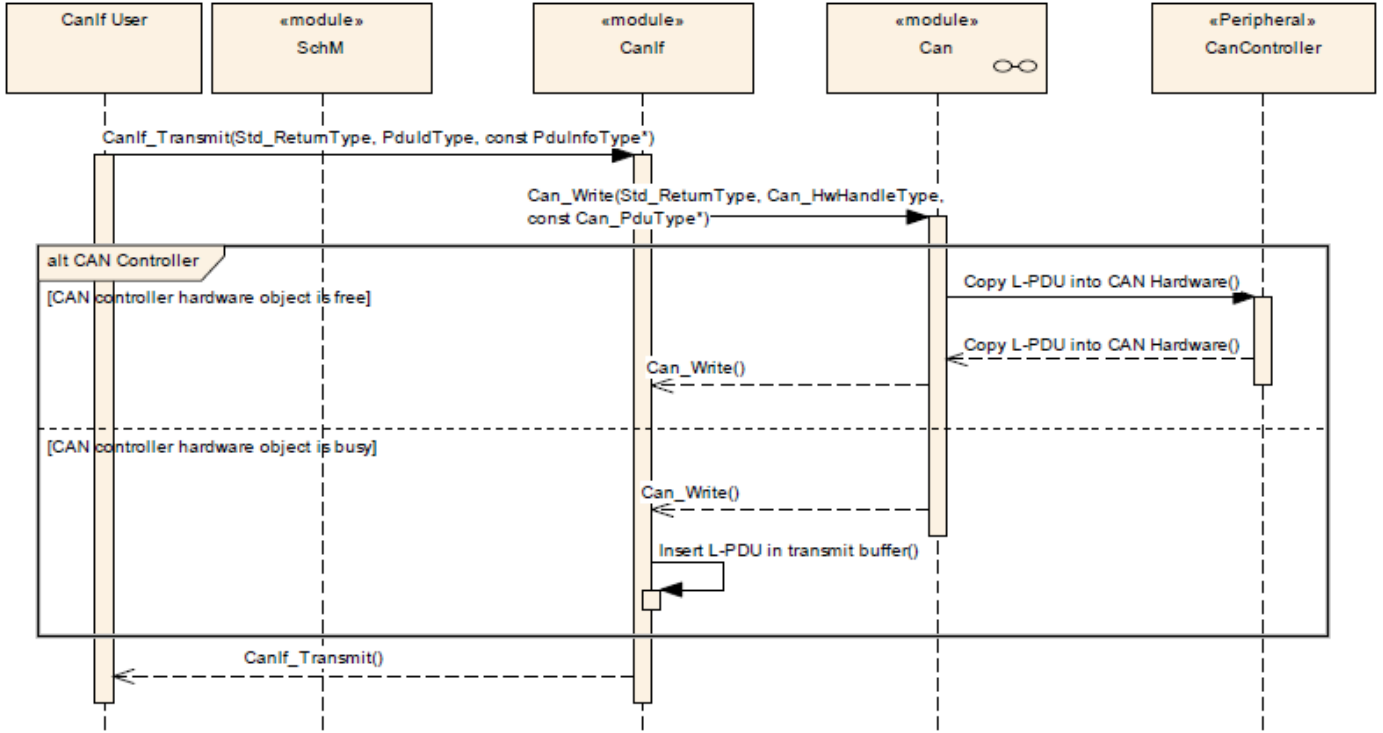


Figure 9.1: Transmission request with a single CAN Driver

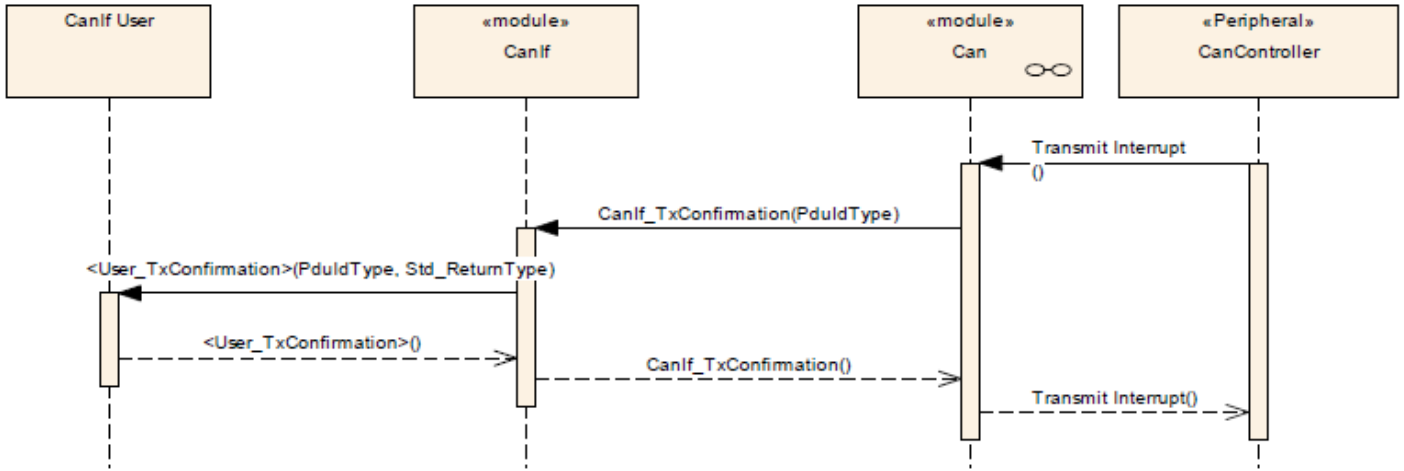


Figure 9.3: Transmit confirmation interrupt driven

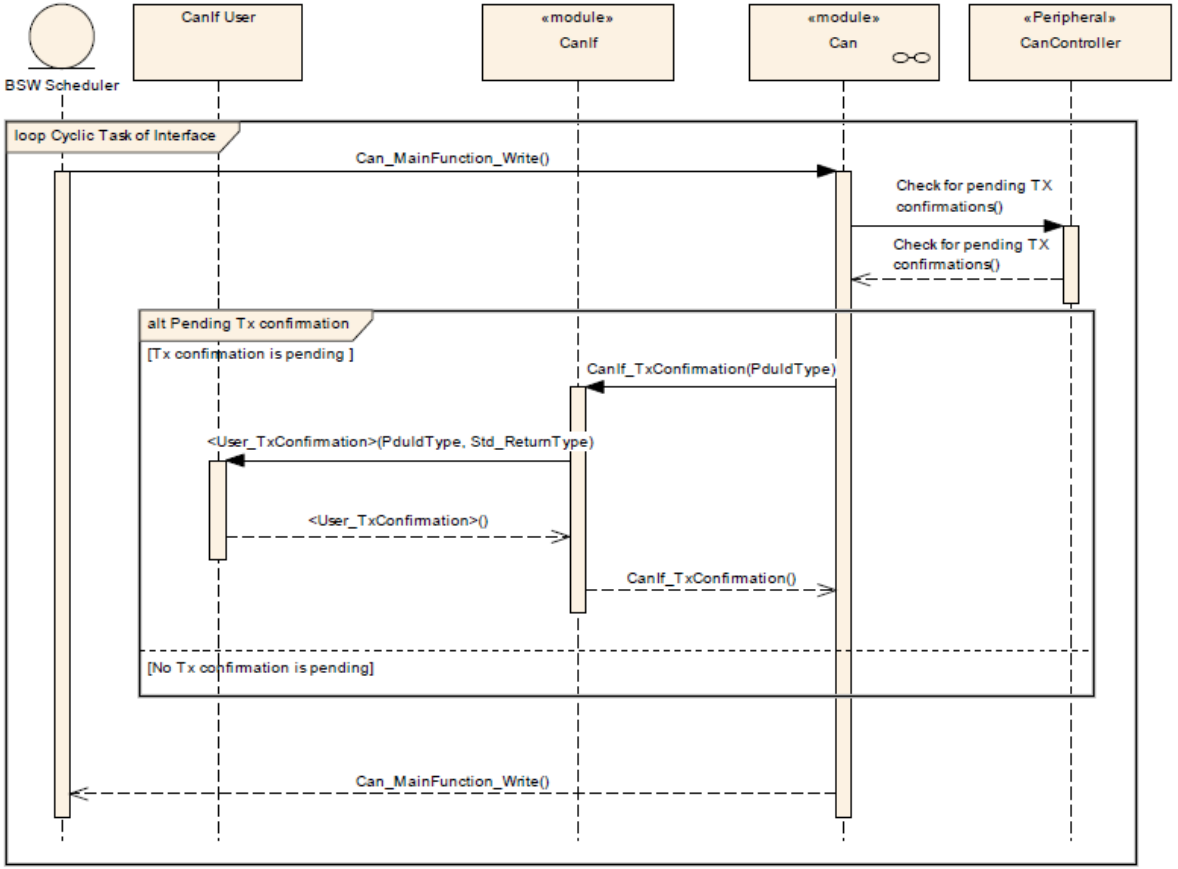


Figure 9.4: Transmit confirmation polling driven

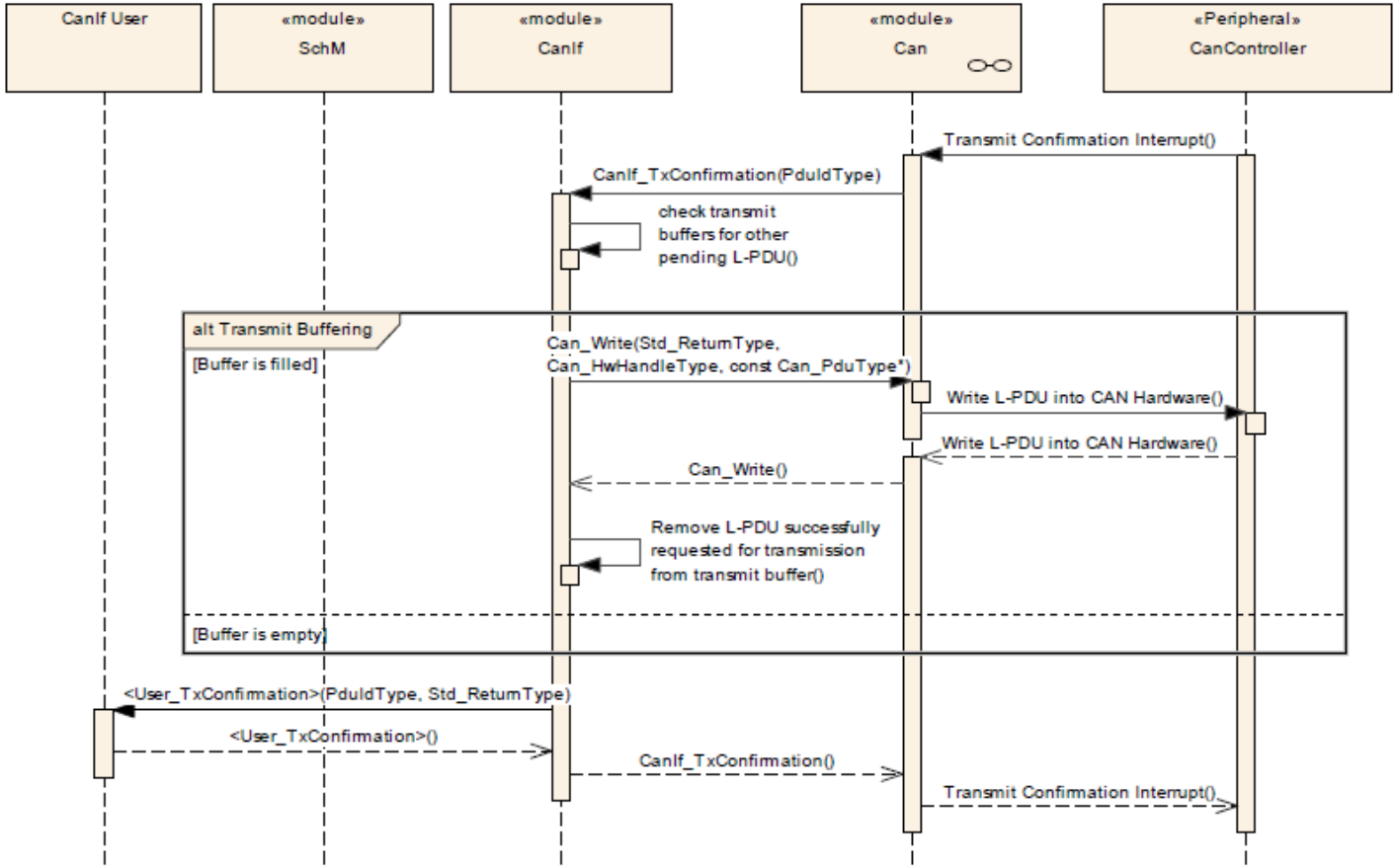


Figure 9.5: Transmit confirmation with buffering

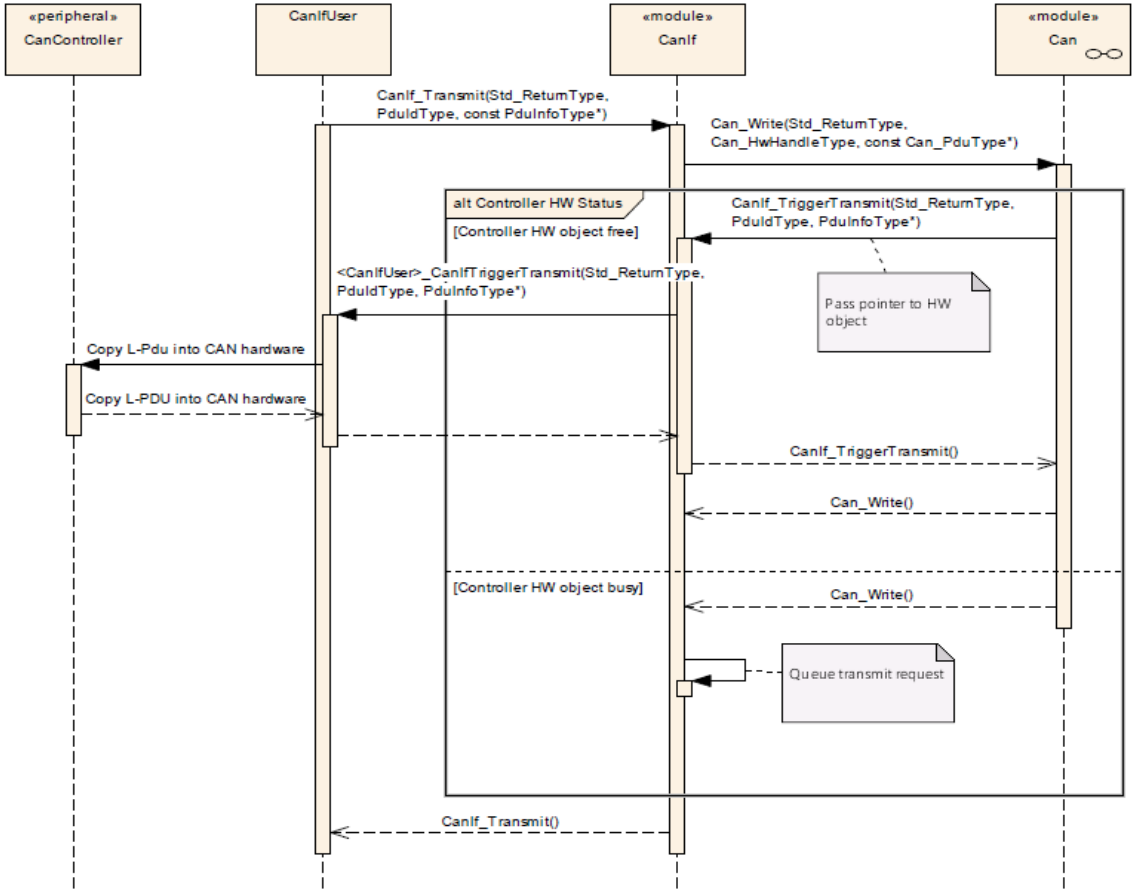


Figure 9.6: Trigger Transmit Request

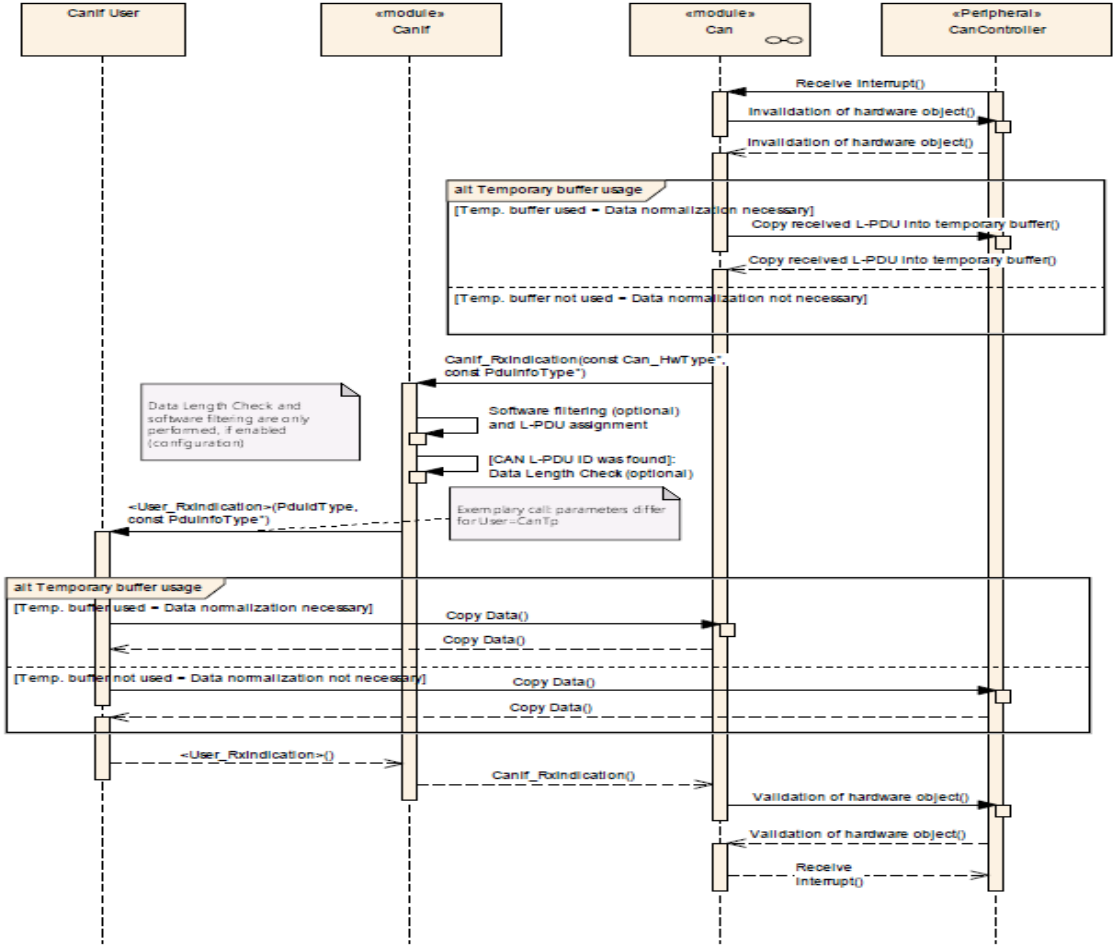


Figure 9.7: Receive indication interrupt driven

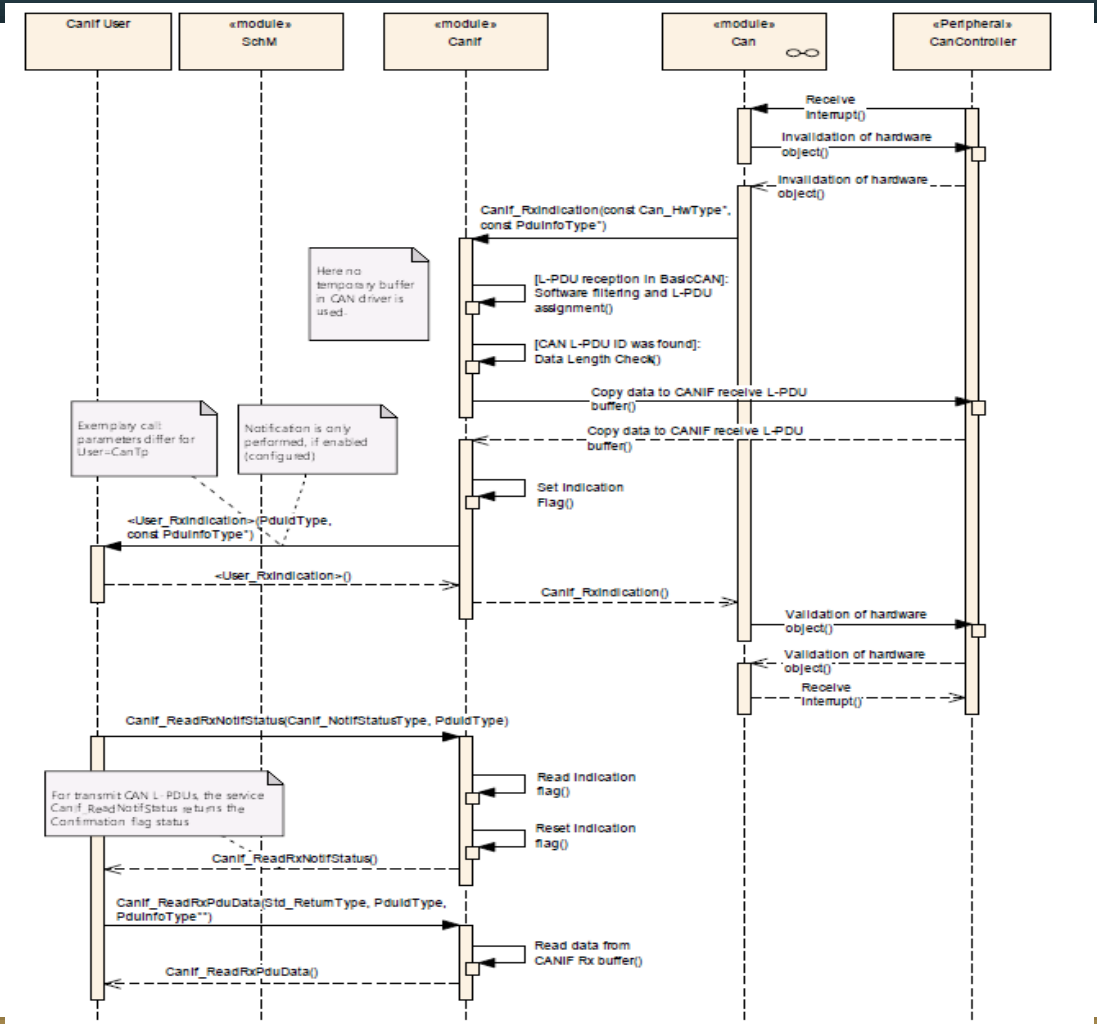


Figure 9.9: Read received data

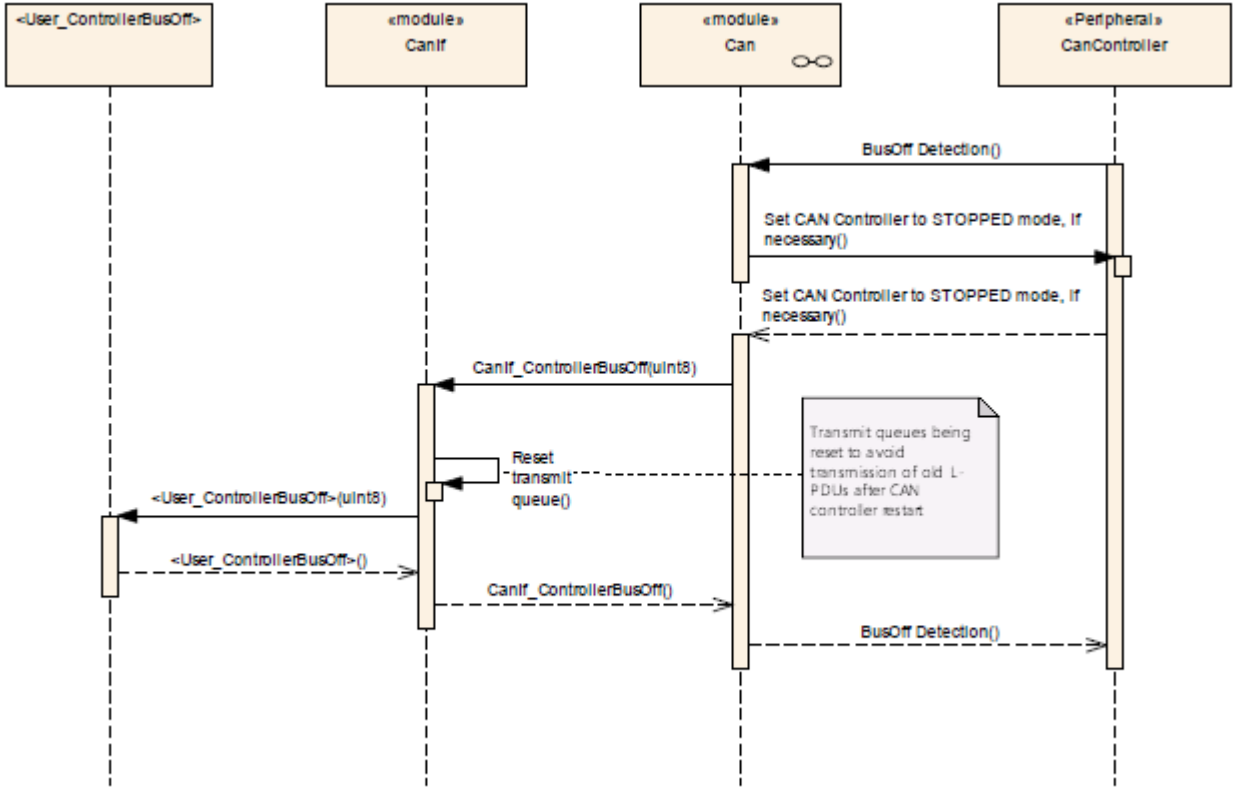


Figure 9.11: BusOff notification

Configuration specification

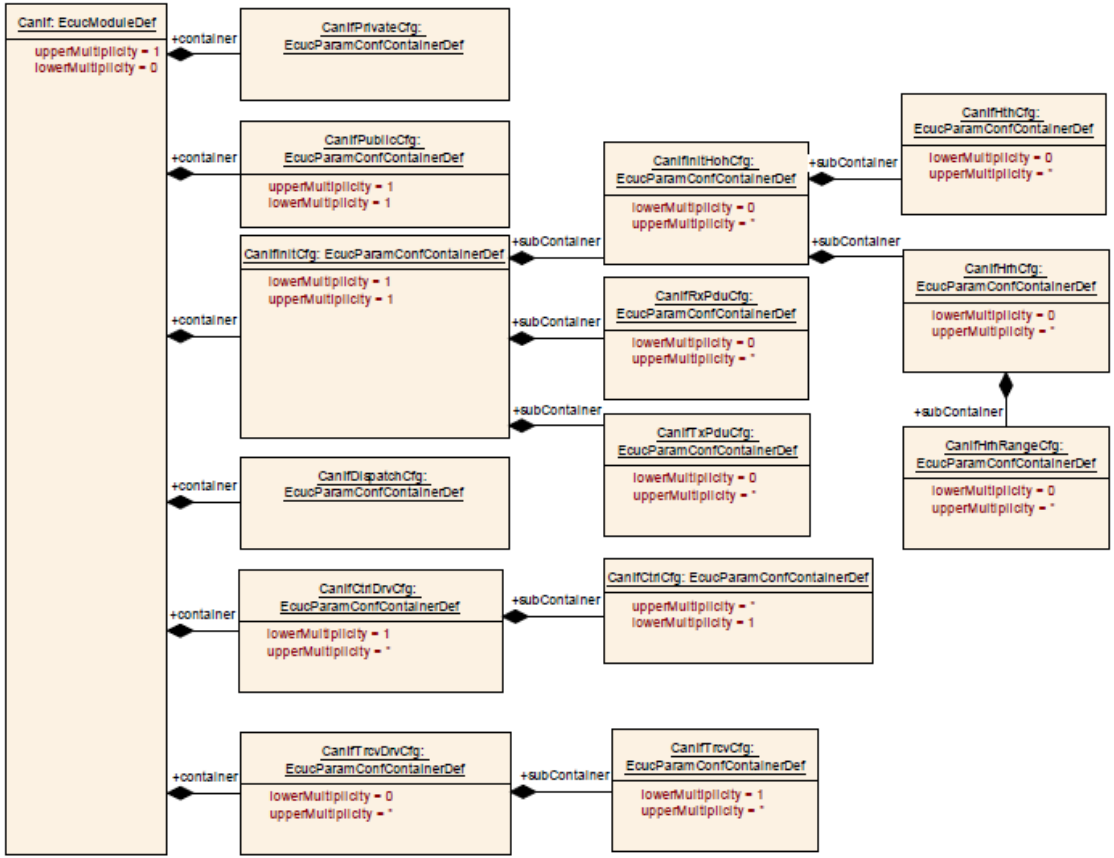


Figure 10.1: Overview about CAN Interface configuration containers

CanIfPrivateCfg

CanIfFixedBuffer	This parameter defines if the buffer element length shall be fixed to 8 Bytes for buffers to which only PDUs < 8 Bytes are assigned.
CanIfPrivateDataLengthCheck	Selects whether Data Length Check is supported.
CanIfPrivateSoftwareFilterType	Selects the desired software filter mechanism for reception only. Each implemented software filtering method is identified by this enumeration number. BINARY/ INDEX/ LINEAR / TABLE
CanIfSupportTTCAN	Defines whether TTCAN is supported.

CanIfPublicCfg

CanIfBusMirroringSupport	Enable support for Bus Mirroring.
CanIfDevErrorDetect	Switches the development error detection and notification on or off.
CanIfEnableSecurityEventReporting	Switches the reporting of security events to the IdsM: - true / false
CanIfGlobalTimeSupport	Enables/Disables the Global Time APIs used when hardware timestamping is supported.
CanIfMetaDataSupport	Enable support for dynamic ID handling using L-SDU MetaData.
CanIfPublicCddHeaderFile	Defines header files for callback functions which shall be included in case of CDDs. Range of characters is 1.. 32.
CanIfPublicHandleTypeEnum	This parameter is used to configure the Can_HwHandleType. The Can_HwHandleType represents the hardware object handles of a CAN hardware unit.
CanIfPublicMultipleDrvSupport	Selects support for multiple CAN Drivers. True/False
CanIfPublicPnSupport	Selects support of Partial Network features in CanIf. True/False
CanIfPublicReadRxPduDataApi	Enables / Disables the API CanIf_ReadRxPduData() for reading received L-SDU data. True/False
CanIfPublicReadRxPduNotifyStatusApi	Enables and disables the API for reading the notification status of receive L-PDUs. True/False
CanIfPublicReadTxPduNotifyStatusApi	Enables and disables the API for reading the notification status of transmit L-PDUs. True/False

CanIfPublicSetDynamicTxIdApi	Enables and disables the API for reconfiguration of the CAN Identifier for each Transmit L-PDU. True/False
CanIfPublicTxBuffering	Enables and disables the buffering of transmit L-PDUs (rejected by the CanDrv) within the CAN Interface module. True/False
CanIfPublicTxConfirmPollingSupport	Configuration parameter to enable/disable the API to poll for Tx Confirmation state.
CanIfPublicWakeupCheckValidByNM	If enabled, only NM messages shall validate a detected wake-up event in CanIf. If disabled, all received messages corresponding to a configured Rx PDU shall validate such a wake-up event.
CanIfPublicWakeupCheckValidSupport	Selects support for wake up validation. True/ False
CanIfSetBaudrateApi	Configuration parameter to enable/disable the CanIf_SetBaudrate API to change the baud rate of a CAN Controller.
CanIfTriggerTransmitSupport	Enables the CanIf_TriggerTransmit API at Pre-Compile-Time. True/False
CanIfTxOfflineActiveSupport	Determines wether TxOffLineActive feature. True/False
CanIfVersionInfoApi	Enables and disables the API for reading the version information about the CAN Interface. True/False
CanIfWakeupSupport	Enables the CanIf_CheckWakeup API at Pre-Compile-Time. True/False

CanIfInitCfg

CanIfInitCfgSet	Selects the CAN Interface specific configuration setup. This type of the external data structure shall contain the post build initialization data for the CAN Interface for all underlying CAN Dirvers.
CanIfMaxBufferSize	Maximum total size of all Tx buffers. This parameter is needed only in case of post-build loadable implementation using static memory allocation.
CanIfMaxRxPduCfg	Maximum number of Pdus. This parameter is needed only in case of post-build loadable implementation using static memory allocation.
CanIfMaxTxPduCfg	Maximum number of Pdus. This parameter is needed only in case of post-build loadable implementation using static memory allocation.
CanIfTxPduCfg	This container contains the configuration (parameters) of a transmit CAN L-PDU. It has to be configured as often as a transmit CAN L-PDU is needed.
CanIfRxPduCfg	This container contains the configuration (parameters) of each receive CAN L-PDU.
CanIfInitHohCfg	This container contains the references to the configuration setup of each underlying CAN Driver.
CanIfBufferCfg	This container contains the Txbuffer configuration. Multiple buffers with different sizes could be configured.

CanIfTxPduCfg

CanIfTxPduCanId	CAN Identifier of transmit CAN L-PDUs used for CAN L-PDU transmission.
CanIfTxPduCanIdMask	Identifier mask which denotes relevant bits in the CAN Identifier. Range: 11 bits for Standard CAN Identifier, 29 bits for Extended CAN Identifier.
CanIfTxPduCanIdType	EXTENDED_CAN/ EXTENDED_FD_CAN / STANDARD_CAN / STANDARD_FD_CAN
CanIfTxPduId	ECU wide unique, symbolic handle for transmit CAN L-SDU.
CanIfTxPduPnFilterPdu	by this parameter PDUs could be configured which will pass the CanIfPnFilter.
CanIfTxPduReadNotifyStatus	Enables and disables transmit confirmation for each transmit CAN. True/ False
CanIfTxPduTriggerTransmit	Determines if or if not CanIf shall use the trigger transmit API for this PDU.
CanIfTxPduTruncation	Enables/disables truncation of PDUs that exceed the configured size.
CanIfTxPduType	Defines the type of each transmit CAN L-PDU. (Dynamic / Static)
CanIfTxPduUserTriggerTransmitName	Defines the name of the <User_TriggerTransmit>. Depends CanIfTxPduUserTxConfirmationUL.
CanIfTxPduUserTxConfirmationName	Defines the name of the <User_TxConfirmation>. >. Depends CanIfTxPduUserTxConfirmationUL.
CanIfTxPduUserTxConfirmationUL	defines the upper layer (UL) module to which the confirmation of the successfully transmitted CanTxPduId has to be routed via the <User_TxConfirmation>.
CanIfTxPduBufferRef	Configurable reference to a CanIf buffer configuration.
CanIfTxPduRef	Reference to the "global" Pdu structure to allow harmonization of handle IDs in the COM-Stack.

CanIfRxPduCfg

CanIfRxPduCanId	CAN Identifier of Receive CAN L-PDUs used by the CAN Interface. Range: 11 Bit For Standard CAN Identifier ... 29 Bit For Extended CAN identifier
CanIfRxPduCanIdMask	Identifier mask which denotes relevant bits in the CAN Identifier.
CanIfRxPduCanIdType	EXTENDED_CAN/ EXTENDED_FD_CAN / STANDARD_CAN / STANDARD_FD_CAN / EXTENDED_NO_FD_CAN / STANDARD_NO_FD_CAN
CanIfRxPduDataLength	Data length of the received CAN L-PDUs used by the CAN Interface.
CanIfRxPduDataLengthCheck	This parameter switches the message specific data length check. True/False
CanIfRxPduId	ECU wide unique, symbolic handle for receive CAN L-SDU.
CanIfRxPduReadData	Enables and disables the Rx buffering for reading of received L-SDU data. True/ False
CanIfRxPduReadNotifyStatus	Enables and disables receive indication for each receive CAN L-SDU for reading its notification status. True/ False
CanIfRxPduUserRxIndicationName	Defines the name of the <User_RxIndication>, depends on CanIfRxPduUserRxIndicationUL.
CanIfRxPduUserRxIndicationUL	defines the upper layer (UL) module to which the indication of the successfully received CANRXPDUID has to be routed via <User_RxIndication>.
CanIfRxPduHrhIdRef	The HRH to which Rx L-PDU belongs to, is referred through this parameter.
CanIfRxPduRef	Reference to the "global" Pdu structure to allow harmonization of handle IDs in the COM-Stack.

CanIfDispatchCfg

CanIfDispatchUserCheckTrcvWakeFlagIndicationName	defines the name of <User_CheckTrcvWakeFlagIndication>.
CanIfDispatchUserCheckTrcvWakeFlagIndicationUL	defines the upper layer module to which the CheckTrcvWakeFlagIndication from the Driver modules have to be routed.
CanIfDispatchUserClearTrcvWufFlagIndicationName	defines the name of <User_ClearTrcvWufFlagIndication>.
CanIfDispatchUserClearTrcvWufFlagIndicationUL	defines the upper layer module to which the ClearTrcvWufFlagIndication from the Driver modules have to be routed
CanIfDispatchUserConfirmPnAvailabilityName	defines the name of <User_ConfirmPnAvailability>.
CanIfDispatchUserConfirmPnAvailabilityUL	defines the upper layer module to which the ConfirmPnAvailability notification from the Driver modules have to be routed.
CanIfDispatchUserCtrlBusOffName	defines the name of <User_ControllerBusOff>.
CanIfDispatchUserCtrlBusOffUL	defines the upper layer (UL) module to which the notifications of all ControllerBusOff events from the CAN Driver modules have to be routed via <User_ControllerBusOff>.
CanIfDispatchUserCtrlModeIndicationName	defines the name of <User_TrcvModeIndication>.

CanIfDispatchUserCtrlModeIndicationUL	defines the upper layer (UL) module to which the notifications of all ControllerTransition events from the CAN Driver modules have to be routed via <User_ControllerModeIndication>.
CanIfDispatchUserValidateWakeupEventName	This parameter defines the name of <User_ValidateWakeupEvent>.
CanIfDispatchUserValidateWakeupEventUL	defines the upper layer (UL) module to which the notifications about positive former requested wake up sources have to be routed via <User_ValidateWakeupEvent>.
CanIfDispatchUserTrcvModeIndicationName	defines the name of <User_TrcvModeIndication>.
CanIfDispatchUserTrcvModeIndicationUL	This parameter defines the upper layer (UL) module to which the notifications of all TransceiverTransition events from the CAN Transceiver Driver modules have to be routed via <User_TrcvModeIndication>. If

CanIfCtrlCfg

CanIfCtrlId	This parameter abstracts from the CAN Driver specific parameter Controller.
CanIfCtrlWakeupSupport	This parameter defines if a respective controller of the referenced CAN Driver modules is queriable for wake up events.
CanIfCtrlCanCtrlRef	This parameter references to the logical handle of the underlying CAN controller from the CAN Driver module to be served by the CAN Interface module.

CanIfCtrlDrvCfg

CanIfCtrlDrvInitHohConfigRef	Reference to the Init Hoh Configuration
CanIfCtrlDrvNameRef	CAN Interface Driver Reference. This reference can be used to get any information (Ex. Driver Name, Vendor ID) from the CAN driver.

CanIfTrcvCfg

CanIfTrcvId	This parameter abstracts from the CAN Transceiver Driver specific parameter Transceiver.
CanIfTrcvWakeupSupport	This parameter defines if a respective transceiver of the referenced CAN Transceiver Driver modules is queriable for wake up events.
CanIfTrcvCanTrcvRef	This parameter references to the logical handle of the underlying CAN transceiver from the CAN transceiver driver module to be served by the CAN Interface module.

CanIfInitHohCfg

CanIfHrhCfg	This container contains configuration parameters for each hardware receive object (HRH).
CanIfHthCfg	This container contains parameters related to each HTH.

CanIfHthCfg

CanIfHthCanCtrlIdRef	Reference to controller Id to which the HTH belongs to. A controller can contain one or more HTHs.
CanIfHthIdSymRef	The parameter refers to a particular HTH object in the CanDrv configuration .CanIf receives the following information of the CanDrv module by this reference: CanHandleType & CanObjectId

CanIfHrhCfg

CanIfHrhSoftwareFilter	Selects the hardware receive objects by using the HRH range/list from CAN Driver configuration to define, for which HRH a software filtering has to be performed at during receive processing.
CanIfHrhCanCtrlIdRef	Reference to controller Id to which the HRH belongs to. A controller can contain one or more HRHs.
CanIfHrhIdSymRef	The parameter refers to a particular HRH object in the CanDrv configuration .CanIf receives the following information of the CanDrv module by this reference: CanHandleType & CanObjectId

CanIfHrhRangeCfg

CanIfHrhRangeBaseId	CAN Identifier used as base value in combination with CanIfHrhRangeMask for a masked ID range in which all CAN Ids shall pass the software filtering.
CanIfHrhRangeMask	Used as mask value in combination with CanIfHrhRangeBaseId for a masked ID range in which all CAN Ids shall pass the software filtering.
CanIfHrhRangeRxPduLowerCanId	Lower CAN Identifier of a receive CAN L-PDU for identifier range definition, in which all CAN Ids shall pass the software filtering.
CanIfHrhRangeRxPduRangeCanIdType	Specifies whether a configured Range of CAN Ids shall only consider standard CAN Ids or extended CAN Ids.
CanIfHrhRangeRxPduUpperCanId	Upper CAN Identifier of a receive CAN L-PDU for identifier range definition, in which all CAN Ids shall pass the software filtering.

CanIfBufferCfg

CanIfBufferSize	This parameter defines the number of CanIf Tx L-PDUs which can be buffered in one Txbuffer.
CanIfBufferHthRef	Reference to HTH, that defines the hardware object or the pool of hardware objects configured for transmission.

CanIfSecurityEventRefs

CANIF_SEV_ERRORSTATE_BUSOFF	The CAN controller transitioned to state busoff.
CANIF_SEV_ERRORSTATE_PASSIVE	A reception related error was detected. Depending on the context data this could indicate suspicious CAN activity.
CANIF_SEV_RX_ERROR_DETECTED	A reception related error was detected. Depending on the context data this could indicate suspicious CAN activity.
CANIF_SEV_TX_ERROR_DETECTED	A transmission related error was detected. Depending on the context data this could indicate suspicious CAN activity.